

Минобрнауки России

Юго-Западный государственный университет

Кафедра программной инженерии

**ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
ПО ПРОГРАММЕ БАКАЛАВРИАТА**

09.03.04 Программная инженерия

(код, наименование ОПОП ВО: направление подготовки, направленность (профиль))

«Разработка программно-информационных систем»

Бизнес-проект "Информационно-аналитическая система для РИО ВУЗа

Разработка сервиса "Учебники и учебные пособия"

(название темы)

Дипломный проект

(вид ВКР: дипломная работа или дипломный проект)

Автор ВКР

С.А. Киреев

(подпись, дата)

(инициалы, фамилия)

Группа ПО-226

Руководитель ВКР

Е. И. Аникина

(подпись, дата)

(инициалы, фамилия)

Нормоконтроль

А. А. Чаплыгин

(подпись, дата)

(инициалы, фамилия)

ВКР допущена к защите:

Заведующий кафедрой

А. В. Малышев

(подпись, дата)

(инициалы, фамилия)

Курск 2026 г.

Минобрнауки России

Юго-Западный государственный университет

Кафедра программной инженерии

УТВЕРЖДАЮ:

Заведующий кафедрой

(подпись, инициалы, фамилия)

« ____ » _____ 20 ____ г.

ЗАДАНИЕ НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ ПО ПРОГРАММЕ БАКАЛАВРИАТА

Студента Киреева С.А., шифр 22-06-0509, группа ПО-22б

1. Тема «Бизнес-проект "Информационно-аналитическая система для РИО ВУЗа Разработка сервиса "Учебники и учебные пособия"» утверждена приказом ректора ЮЗГУ от «03» апреля 2026 г. № 1585-с.

2. Срок предоставления работы к защите «09» июня 2026 г.

3. Исходные данные для создания программной системы:

3.1. Перечень решаемых задач:

1) Провести анализ процессов проверки и оформления учебно-методических материалов в редакционно-издательском отделе.

2) Исследовать существующие подходы к автоматизации проверки документов и генерации титульных листов.

3) Разработать архитектуру информационно-аналитической системы на основе клиент-серверного взаимодействия.

4) Реализовать серверную часть системы для автоматической проверки DOCX-документов, генерации титульных листов и формирования PDF-отчётов.

5) Разработать клиентскую часть веб-приложения для взаимодействия пользователей с системой.

6) Реализовать механизм аутентификации и разграничения прав доступа пользователей.

7) Провести тестирование разработанной информационно-аналитической системы и оценить корректность её работы.

3.2. Входные данные и требуемые результаты для программы:

1) Входными данными для информационно-аналитической системы являются DOCX-документы учебно-методических материалов, параметры генерации титульных листов, а также данные пользователей, вводимые через веб-интерфейс системы.

2) Выходными данными являются результаты автоматической проверки документов, сформированные PDF-отчёты с перечнем ошибок, DOCX-файлы титульных листов и статистическая информация о выполненных проверках.

4. Содержание работы (по разделам):

4.1. Введение.

4.2. Анализ предметной области.

4.3. Техническое задание: основание для разработки, назначение разработки, требования к программной системе, требования к оформлению документации.

4.4. Технический проект: общие сведения о программной системе, проект данных программной системы, проектирование архитектуры программной системы, проектирование пользовательского интерфейса программной системы.

4.5. Рабочий проект: спецификация компонентов и классов программной системы, тестирование программной системы, сборка компонентов программной системы.

4.6. Заключение.

4.7. Список использованных источников.

5. Перечень графического материала:

Лист 1. Сведения о ВКРБ.

Лист 2. Цели и задачи разработки.

- Лист 3. Диаграмма прецедентов пользователя.
- Лист 4. Общая схема архитектуры.
- Лист 5. Архитектура серверной части.
- Лист 6. Схема последовательности аутентификации.
- Лист 7. Структура маршрутизации серверной части.
- Лист 8. Страница авторизации в приложении.
- Лист 9. Страница проверки файлов.
- Лист 10. Страница профиля пользователя.
- Лист 11. Заключение.

Руководитель ВКР

(подпись, дата)

Е. И. Аникина

(инициалы, фамилия)

Задание принял к исполнению

(подпись, дата)

С.А. Киреев

(инициалы, фамилия)

РЕФЕРАТ

Объем работы равен 113 страницам. Работа содержит 17 иллюстраций, 12 таблиц и 54 библиографических источников. Количество приложений – 2. Графический материал представлен в приложении А. Фрагменты исходного кода представлены в приложении Б.

Перечень ключевых слов: Python, REST API, FastAPI, PostgreSQL, информационно-аналитическая система, проверка учебно-методических материалов, веб-приложение, генерация титульных листов, PDF-отчёты.

Объектом разработки является информационно-аналитическая система автоматизации проверки и обработки учебно-методических материалов редакционно-издательского отдела.

Целью работы является разработка информационно-аналитической системы для автоматизации процессов проверки, оформления и сопровождения учебно-методических документов.

Разработанная система реализована на языке Python с использованием web-фреймворка FastAPI и СУБД PostgreSQL. Клиентская часть разработана с применением HTML, CSS и JavaScript. Взаимодействие между компонентами системы осуществляется через REST API.

Архитектура системы построена по клиент-серверной модели. Серверная часть выполняет обработку запросов пользователей, проверку DOCX-документов, генерацию титульных листов, формирование PDF-отчётов и взаимодействие с базой данных. Клиентская часть предоставляет web-интерфейс для загрузки документов и просмотра результатов проверки.

В системе реализован модуль автоматической проверки DOCX-документов, выполняющий анализ параметров форматирования и структуры учебно-методических материалов. Результаты проверки используются для формирования PDF-отчётов с перечнем найденных ошибок.

Разработанная система позволяет сократить время проверки документов, уменьшить количество ошибок оформления и повысить эффективность работы редакционно-издательского отдела.

ABSTRACT

The volume of work is 113 pages. The work contains 17 illustrations, 12 tables and 54 bibliographic sources. The number of applications is 2. The graphic material is presented in Appendix A. Fragments of the source code are presented in Appendix B.

List of keywords: Python, REST API, FastAPI, PostgreSQL, information and analytical system, verification of teaching materials, web application, generation of cover pages, PDF reports.

The object of the development is an information and analytical automation system for checking and processing educational and methodological materials of the editorial and publishing department.

The purpose of the work is to develop an information and analytical system for automating the processes of verification, registration and maintenance of educational and methodological documents.

The developed system is implemented in Python using the FastAPI web framework and the PostgreSQL database management system. The client side is developed using HTML, CSS, and JavaScript. The interaction between the system components is carried out through the REST API.

The architecture of the system is based on the client-server model. The server side processes user requests, validates DOCX documents, generates title pages, generates PDF reports, and interacts with the database. The client side provides a web interface for downloading documents and viewing the verification results.

The system implements an automatic verification module for DOCX documents, which analyzes the formatting parameters and structure of teaching materials. The verification results are used to generate PDF reports with a list of errors found.

The developed system makes it possible to reduce the time needed to check documents, reduce the number of design errors, and improve the efficiency of the editorial and publishing department.

СОДЕРЖАНИЕ

РЕЗЮМЕ СТАРТАП-ПРОЕКТА	9
ВВЕДЕНИЕ	12
1 Анализ предметной области	15
1.1 Характеристика деятельности редакционно-издательского отдела вуза	15
1.2 Проблематика традиционного подхода к подготовке к изданию методических материалов	16
1.3 Обоснование необходимости автоматизации	17
1.4 Анализ существующих систем автоматизации документооборота и проверки документов	18
1.4.1 Microsoft Word и встроенные средства рецензирования	18
1.4.2 Google Docs	18
1.4.3 Системы электронного документооборота	19
1.4.4 Вывод по анализу аналогов	19
1.5 Постановка задач на разработку	20
2 Техническое задание	21
2.1 Основание для разработки	21
2.2 Цель и назначение разработки	21
2.3 Требования к функциональности	21
2.4 Требования к интерфейсу	22
2.5 Требования к обработке ошибок	26
2.6 Требования к удобству использования (UX)	27
2.7 Моделирование вариантов использования	27
2.7.1 Вариант использования «Вход в систему»	30
2.7.2 Вариант использования «Выход из системы»	31
2.7.3 Вариант использования «Загрузка документа на проверку»	31
2.7.4 Вариант использования «Автоматическая проверка документа»	32
2.7.5 Вариант использования «Получение отчёта об ошибках»	32

2.7.6	Вариант использования «Скачивание документа с выделенными ошибками»	33
2.7.7	Вариант использования «Автоматическое сохранение корректного документа»	33
2.7.8	Вариант использования «Просмотр архива документов»	34
2.7.9	Вариант использования «Формирование титульного листа»	34
2.7.10	Вариант использования «Просмотр статистики»	35
2.7.11	Вариант использования «Управление пользователями»	35
2.7.12	Вариант использования «Настройка параметров системы»	35
2.8	Требования к надёжности	36
2.9	Требования к информационной безопасности	37
2.10	Требования к программной и технической совместимости	37
2.11	Требования к оформлению документации	38
3	Технический проект	39
3.1	Архитектура программной системы	39
3.2	Обоснование выбора технологии проектирования	41
3.2.1	Язык программирования Python	41
3.2.2	Веб-фреймворк FastAPI	42
3.2.3	СУБД PostgreSQL	43
3.2.4	Средства работы с документами	44
3.2.5	Прочие библиотеки и инструменты	45
3.3	Описание серверной части	46
3.4	Описание API	49
3.4.1	Примеры JSON-запросов	53
3.5	Модуль маршрутизации и обработки HTTP-запросов	56
3.6	Модуль аутентификации и авторизации	57
3.6.1	Процесс аутентификации пользователя	57
3.6.2	JWT-токены	60
3.6.3	Хеширование паролей	60
3.6.4	Роли пользователей и разграничение прав доступа	60
3.6.5	Хранение refresh token	61

3.6.6	Dependencies и middleware	62
3.7	Модуль проверки документов	62
3.7.1	Основные функции модуля проверки	63
3.8	Модуль генерации титульных листов	63
3.8.1	Назначение модуля	63
3.8.2	Генерация DOCX-документов	64
3.8.3	Форматирование титульного листа	64
3.9	Модуль формирования отчётов	64
3.9.1	Основные функции модуля	64
3.9.2	Группировка ошибок	65
3.9.3	Структура PDF-отчёта	65
3.9.4	Взаимодействие с компонентами системы	65
3.10	Модуль работы с базой данных	66
3.11	Форматы данных	68
4	Рабочий проект	70
4.1	Спецификация классов программной системы	70
4.1.1	Класс User (модель пользователя)	70
4.1.2	Класс Faculty (модель факультета)	71
4.1.3	Класс Department (модель кафедры)	71
4.1.4	Класс Manual (модель методических указаний)	72
4.1.5	Класс Tutorial (модель учебного пособия)	72
4.1.6	Класс Monograph (модель монографии)	73
4.1.7	Модуль проверки документов	73
4.1.8	Модуль генерации титульных листов	75
4.1.9	Модуль формирования PDF-отчётов	76
4.2	Системное тестирование	77
4.2.1	Сводная таблица тест-кейсов	77
4.2.2	ТС-01. Генерация титульного листа учебного издания	78
4.2.3	ТС-02. Автоматическая проверка DOCX-документа	79
4.2.4	ТС-03. Формирование PDF-отчёта о результатах проверки	80
4.2.5	ТС-04. Скачивание сформированного PDF-отчёта	81

4.2.6	Результаты тестирования	81
4.2.7	Нефункциональное тестирование	82
4.2.8	Вывод по результатам тестирования	83
	ЗАКЛЮЧЕНИЕ	84
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	85
	ПРИЛОЖЕНИЕ А Представление графического материала	91
	ПРИЛОЖЕНИЕ Б Фрагменты исходного кода программы	103
	На отдельных листах (CD-RW в прикрепленном конверте)	113
	Сведения о ВКРБ (Графический материал / Сведения о ВКРБ.png)	Лист 1
	Цели и задачи разработки (Графический материал / Цели и задачи разработки.png)	Лист 2
	Диаграмма прецедентов пользователя (Графический материал / Диаграмма прецедентов пользователя.png)	Лист 3
	Общая схема архитектуры (Графический материал / Общая схема архитектуры.png)	Лист 4
	Архитектура серверной части (Графический материал / Архитектура серверной части.png)	Лист 5
	Схема последовательности аутентификации (Графический материал / Схема последовательности аутентификации.png)	Лист 6
	Структура маршрутизации серверной части (Графический материал / Структура маршрутизации серверной части.png)	Лист 7
	Страница авторизации в приложении (Графический материал / Страница авторизации в приложении.png)	Лист 8
	Страница проверки файлов (Графический материал / Страница проверки файлов.png)	Лист 9
	Страница профиля пользователя (Графический материал / Страница профиля пользователя.png)	Лист 10
	Заключение (Графический материал / Заключение.png)	Лист 11

ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

РИО — редакционно-издательский отдел.

БД — база данных.

СУБД — система управления базами данных.

ORM (Object–Relational Mapping) — технология объектно-реляционного отображения данных.

REST API (Representational State Transfer Application Programming Interface) — программный интерфейс взаимодействия компонентов системы.

JWT (JSON Web Token) — формат токена для аутентификации и авторизации пользователей.

PDF (Portable Document Format) — формат электронных документов.

DOCX — формат текстовых документов Microsoft Word.

HTML (HyperText Markup Language) — язык гипертекстовой разметки веб-страниц.

CSS (Cascading Style Sheets) — язык описания внешнего вида веб-страниц.

SQL (Structured Query Language) — язык структурированных запросов к базе данных.

FastAPI — веб-фреймворк для разработки серверной части приложения на языке Python.

PostgreSQL — объектно-реляционная система управления базами данных.

CRUD (Create, Read, Update, Delete) — основные операции обработки данных.

GUI (Graphical User Interface) — графический пользовательский интерфейс. API (Application Programming Interface) — программный интерфейс взаимодействия компонентов системы.

РЕЗЮМЕ СТАРТАП-ПРОЕКТА

Название: «Информационно-аналитическая система автоматизации проверки и оформления учебно-методических материалов редакционно-издательского отдела». Проект представляет собой web-ориентированную систему, предназначенную для автоматизации процессов проверки, оформления и обработки учебно-методических документов в редакционно-издательском отделе образовательной организации.

Система обеспечивает автоматическую проверку DOCX-документов на соответствие требованиям оформления, генерацию титульных листов, формирование PDF-отчётов о найденных ошибках, а также централизованное хранение и обработку результатов проверки.

Ключевые возможности системы включают:

1. Автоматическую проверку учебно-методических материалов на соответствие установленным требованиям оформления с выделением найденных ошибок.
2. Генерацию титульных листов для методических указаний, учебных пособий и монографий на основе пользовательских данных.
3. Формирование PDF-отчётов с результатами проверки документов и статистикой найденных нарушений.
4. Реализацию системы аутентификации и разграничения прав доступа пользователей.
5. Ведение статистики проверок и централизованное хранение информации о документах и пользователях.
6. Интуитивно понятный web-интерфейс, обеспечивающий удобную работу пользователей с системой.

Разработанная система позволяет существенно сократить время проверки учебно-методических материалов, уменьшить количество ошибок оформления и повысить эффективность работы редакционно-издательского отдела.

Актуальность проекта обусловлена необходимостью автоматизации процессов проверки и оформления документов в образовательных организациях. Традиционная ручная проверка требует значительных временных затрат и сопровождается высокой вероятностью появления ошибок, связанных с человеческим фактором.

Целью разработки является создание информационно-аналитической системы, обеспечивающей:

- автоматизированную проверку учебно-методических материалов;
- автоматическое формирование титульных листов и сопроводительной документации;
- повышение точности проверки документов;
- сокращение времени обработки учебных изданий;
- централизованное управление процессами проверки и хранения документов.

Отличительные особенности разработанной системы по сравнению с существующими решениями:

1. Комплексная автоматизация процессов проверки и оформления учебно-методических материалов в рамках единой системы.
2. Автоматическое формирование PDF-отчётов с визуализацией найденных ошибок.
3. Поддержка генерации титульных листов для различных типов учебных изданий.
4. Использование клиент-серверной архитектуры, обеспечивающей возможность масштабирования системы.
5. Гибкая система управления пользователями и разграничения прав доступа.

В системе реализованы функции автоматической проверки DOCX-документов, генерации титульных листов, формирования PDF-отчётов, ведения статистики проверок, а также механизмы авторизации и взаимодействия клиентской и серверной части через REST API.

Идея создания проекта возникла в 2025 году в связи с необходимостью автоматизации процессов проверки учебно-методических материалов в редакционно-издательском отделе образовательной организации. Разработка системы была начата в 2026 году в рамках выполнения выпускной квалификационной работы.

Бизнес-цели:

1. Снижение временных затрат на проверку и оформление учебно-методических материалов.
2. Повышение качества и единообразия оформления документов.
3. Сокращение количества ошибок, связанных с ручной проверкой документов.
4. Повышение эффективности взаимодействия сотрудников редакционно-издательского отдела и преподавателей.
5. Возможность дальнейшего расширения функциональности системы и интеграции с внутренними информационными системами образовательной организации.

ВВЕДЕНИЕ

В условиях постоянного роста объёмов учебно-методической документации особую актуальность приобретает автоматизация процессов проверки, оформления и сопровождения документов. В образовательных организациях значительное количество времени сотрудников редакционно-издательских отделов затрачивается на ручную проверку учебных пособий, методических указаний и других материалов на соответствие установленным требованиям оформления. Традиционный подход приводит к увеличению трудозатрат, повышает вероятность появления ошибок и усложняет контроль качества документации.

Современные информационные технологии позволяют автоматизировать процессы обработки документов, обеспечить централизованное хранение информации и повысить эффективность взаимодействия между пользователями системы. Веб-ориентированные информационно-аналитические системы предоставляют возможность автоматизированной проверки документов, генерации отчётов, формирования титульных листов и организации взаимодействия между авторами и сотрудниками редакционно-издательского отдела.

Разработанная информационно-аналитическая система позволяет автоматизировать процесс проверки учебно-методических материалов, сократить время обработки документов, снизить влияние человеческого фактора и обеспечить единые стандарты оформления. Система даёт возможность авторам загружать документы, получать детальные отчёты об ошибках, скачивать исправленные версии с визуальным выделением нарушений, а также формировать титульные листы по утверждённым шаблонам. Сотрудники редакционно-издательского отдела получают инструмент для централизованного накопления информации о проверенных документах и формирования статистической отчётности.

Цель настоящей работы – разработка информационно-аналитической системы для автоматизации процессов проверки и обработки учебно-методических документов редакционно-издательского отдела.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Изучить особенности организации работы редакционно-издательского отдела и процессов проверки документов.
2. Проанализировать существующие средства автоматизации документооборота и проверки документации.
3. Разработать архитектуру информационно-аналитической системы.
4. Реализовать серверную часть системы с использованием FastAPI и PostgreSQL.
5. Разработать клиентскую часть веб-приложения.
6. Реализовать механизмы проверки документов и анализа параметров форматирования.
7. Разработать модули генерации титульных листов и формирования PDF-отчётов.
8. Провести тестирование разработанной системы и оценить корректность её работы.

Структура и объём работы. Отчёт состоит из введения, четырёх разделов основной части, заключения, списка использованных источников и одного приложения. Текст выпускной квалификационной работы равен 113 страницам.

Во введении сформулирована цель работы, поставлены задачи разработки, описана структура работы, приведено краткое содержание каждого из разделов.

В первом разделе на стадии анализа предметной области приводится характеристика деятельности редакционно-издательского отдела, выявляются проблемы традиционного подхода к проверке материалов, обосновывается необходимость автоматизации, анализируются существующие системы и формулируются задачи на разработку.

Во втором разделе на стадии технического задания приводятся требования к разрабатываемой системе: функциональные требования, требования к интерфейсу, к обработке ошибок, к удобству использования, к надёжности, информационной безопасности и совместимости. Также представлены модели вариантов использования.

В третьем разделе на стадии технического проектирования представлены архитектура программной системы, обоснование выбора технологий, описание модулей проекта).

В четвёртом разделе на стадии рабочего проекта приводится спецификация компонентов серверной части результаты системного и нефункционального тестирования.

В заключении излагаются основные результаты работы, полученные в ходе разработки.

В приложении А представлен графический материал (рисунки, диаграммы).

1 Анализ предметной области

1.1 Характеристика деятельности редакционно-издательского отдела вуза

Редакционно-издательский отдел (РИО) высшего учебного заведения является структурным подразделением, обеспечивающим подготовку, проверку, оформление и выпуск учебно-методической, научной и справочной литературы, используемой в образовательном процессе. К числу основных материалов, поступающих в РИО, относятся методические указания, учебные пособия, практикумы, монографии, сборники научных трудов и иные издания.

Современный образовательный процесс требует постоянного обновления учебно-методической базы. Изменение федеральных государственных образовательных стандартов, развитие научных направлений, внедрение новых технологий обучения и цифровых платформ обуславливают регулярную подготовку новых материалов преподавателями университета.

РИО выполняет функции организационного и нормативного сопровождения издательской деятельности вуза. Сотрудники подразделения принимают материалы от авторов, проверяют их соответствие установленным требованиям оформления, контролируют наличие обязательных структурных элементов, подготавливают замечания, формируют титульные листы, а также ведут учёт поступивших и утверждённых работ.

В условиях большого количества авторов и поступающих документов особую значимость приобретают скорость обработки материалов, точность проверки и удобство взаимодействия между преподавателями и сотрудниками отдела. Эффективность деятельности РИО напрямую влияет на своевременное обеспечение образовательного процесса актуальными учебными материалами.

1.2 Проблематика традиционного подхода к подготовке к изданию методических материалов

Традиционный подход к работе редакционно-издательского отдела, основанный на ручной обработке документов, сопровождается рядом существенных недостатков, снижающих эффективность работы подразделения:

1. Высокая трудоёмкость проверки оформления. В большинстве случаев сотрудники РИО вручную проверяют поступившие документы на соответствие установленным требованиям: параметры шрифта, интервалы, отступы, структура заголовков, оформление списков литературы, таблиц, рисунков и других элементов. При большом количестве работ такой процесс занимает значительное время.

2. Высокая вероятность человеческих ошибок. Ручная проверка не гарантирует обнаружение всех нарушений оформления. Из-за большого объёма текста, однотипности операций и человеческого фактора часть ошибок может остаться незамеченной, что приводит к возврату материалов на доработку уже на поздних этапах.

3. Длительное взаимодействие с авторами. После выявления замечаний сотрудник вынужден вручную составлять перечень ошибок либо вносить исправления в режиме рецензирования документа. Это увеличивает сроки согласования и повторной подачи материалов.

4. Отсутствие централизованного хранилища работ. Готовые и поступившие материалы часто хранятся в отдельных папках, на персональных компьютерах или сетевых дисках. Это затрудняет поиск ранее утверждённых изданий, ведение архива и получение статистических сведений.

5. Сложность формирования отчётности. Для подготовки отчётов по количеству проверенных работ, числу авторов, срокам обработки и видам материалов требуется вручную собирать данные из различных источников. Это увеличивает нагрузку на сотрудников и снижает оперативность получения информации.

6. Отсутствие унифицированных шаблонов. Авторы нередко готовят титульные листы и иные элементы документа самостоятельно, что приводит к многочисленным отклонениям от утверждённых образцов и повторным исправлениям.

Таким образом, ручной подход к организации работы РИО не обеспечивает необходимой скорости, точности и удобства обработки документов в современных условиях.

1.3 Обоснование необходимости автоматизации

Решение перечисленных проблем возможно путём внедрения специализированной информационно-аналитической системы для автоматизации деятельности редакционно-издательского отдела вуза.

Разрабатываемая система должна обеспечить:

- загрузку пользователями учебно-методических и научных материалов через веб-интерфейс;
- автоматическую проверку документов на соответствие требованиям оформления;
- формирование подробного отчёта с перечнем найденных ошибок;
- автоматическое создание файла формата DOCX с выделенными нарушениями;
- автоматическое сохранение корректно оформленных работ в централизованном хранилище;
- ведение статистики поступивших, проверенных и утверждённых материалов;
- генерацию титульных листов по утверждённым шаблонам;
- разграничение прав доступа между авторами, редакторами и администраторами системы;
- удобный интерфейс взаимодействия пользователей с системой.

Автоматизация позволит существенно сократить время проверки материалов, снизить количество ошибок, ускорить процесс согласования документов и обеспечить прозрачный контроль деятельности подразделения.

1.4 Анализ существующих систем автоматизации документооборота и проверки документов

На современном рынке представлены различные программные решения, связанные с управлением документами, совместной работой и проверкой текстовых материалов. Однако большинство из них не ориентировано на специфику работы редакционно-издательского отдела вуза.

1.4.1 Microsoft Word и встроенные средства рецензирования

Текстовый редактор Microsoft Word широко применяется для подготовки и проверки документов. Он предоставляет следующие возможности:

- создание и редактирование документов;
- проверка орфографии и грамматики;
- режим рецензирования и внесения исправлений;
- использование шаблонов оформления.

Недостатком данного подхода является отсутствие автоматизированной проверки документа на соответствие локальным требованиям конкретного вуза. Проверка структуры, параметров оформления и соответствия внутренним стандартам выполняется вручную.

1.4.2 Google Docs

Google Docs является облачным сервисом для совместной работы с документами. Его возможности включают:

- онлайн-редактирование файлов;
- совместную работу нескольких пользователей;
- систему комментариев и предложений;
- хранение документов в облаке.

Несмотря на удобство совместного редактирования, сервис не предоставляет специализированных механизмов автоматической проверки учебно-методических материалов по требованиям университета, а также не содержит функций аналитической отчётности для РИО.

1.4.3 Системы электронного документооборота

Корпоративные системы электронного документооборота предназначены для регистрации, маршрутизации и хранения документов. Они позволяют:

- вести архив документов;
- организовывать согласование файлов;
- разграничивать права доступа;
- контролировать исполнение задач.

Однако подобные системы являются универсальными решениями и не включают инструменты интеллектуального анализа оформления документов DOCX, автоматического подчёркивания ошибок и генерации титульных листов.

1.4.4 Вывод по анализу аналогов

Проведённый анализ существующих решений показал, что универсальные офисные продукты и системы электронного документооборота способны решать лишь отдельные задачи редакционно-издательского отдела.

Одни системы обеспечивают редактирование документов, другие — хранение файлов и организацию согласования, однако ни одно из рассмотренных решений не сочетает в себе функции автоматической проверки оформления методических материалов, генерации отчётов об ошибках, формирования исправленного DOCX-файла и накопления статистики работы подразделения.

Следовательно, наиболее целесообразным является создание собственной специализированной информационно-аналитической системы, учитывающей особенности документооборота и нормативные требования конкретного высшего учебного заведения.

1.5 Постановка задач на разработку

На основании проведённого анализа были сформулированы следующие задачи, которые должна решать разрабатываемая система:

1. Обеспечить регистрацию и авторизацию пользователей с разграничением прав доступа.
2. Реализовать загрузку методических, учебных и научных материалов в формате DOCX.
3. Выполнить автоматическую проверку документов на соответствие установленным требованиям оформления.
4. Формировать отчёт с перечнем найденных нарушений.
5. Создавать копию документа DOCX с визуальным выделением ошибок.
6. Автоматически сохранять корректно оформленные работы в централизованной базе данных.
7. Реализовать хранение и поиск ранее загруженных материалов.
8. Предоставить средства формирования статистической и аналитической отчётности.
9. Реализовать модуль автоматического создания титульных листов по шаблону вуза.
10. Создать удобный веб-интерфейс для пользователей и административную панель для управления системой.

2 Техническое задание

2.1 Основание для разработки

Основанием для разработки является задание на выпускную квалификационную работу бакалавра "Информационно-аналитическая система для РИО ВУЗа. Разработка сервиса "Учебники и учебные пособия".

2.2 Цель и назначение разработки

Основной целью работы является разработка и внедрение информационно-аналитической системы для автоматизации процессов проверки, хранения и сопровождения учебно-методических и научных материалов, поступающих в редакционно-издательский отдел вуза.

Информационно-аналитическая система предназначена для преподавателей, являющихся авторами учебно-методических разработок и сотрудников редакционно-издательского отдела вуза.

2.3 Требования к функциональности

Информационная система должна обеспечивать выполнение следующих функций:

1. Управление пользователями. Система должна обеспечивать регистрацию и авторизацию пользователей по логину и паролю. Должно быть реализовано разграничение прав доступа: администратор, редактор, пользователь. Администратор должен иметь возможность управления учётными записями пользователей.

2. Загрузка документов. Пользователь должен иметь возможность загружать методические материалы, учебные пособия, монографии и иные документы в формате DOCX через веб-интерфейс системы.

3. Автоматическая проверка оформления. После загрузки документа система должна автоматически анализировать файл на соответствие установленным требованиям оформления. Проверка должна включать контроль

структуры документа, параметров шрифта, интервалов, отступов, заголовков, таблиц, изображений, списков литературы и иных элементов.

4. Формирование отчёта об ошибках. По завершении анализа система должна формировать отчёт, содержащий перечень обнаруженных нарушений, описание ошибок и рекомендации по их устранению.

5. Создание документа с выделением ошибок. Система должна создавать копию загруженного документа в формате DOCX, в которой найденные ошибки визуально выделены средствами форматирования.

6. Хранение корректных документов. Если загруженный документ соответствует всем требованиям, он должен автоматически сохраняться в централизованной базе данных или файловом архиве системы.

7. Поиск и просмотр документов. Система должна обеспечивать просмотр списка ранее загруженных материалов, поиск по названию, автору, дате загрузки, статусу проверки и другим параметрам.

8. Генерация титульных листов. Пользователь должен иметь возможность заполнить форму с исходными данными, после чего система должна автоматически сформировать титульный лист по утверждённому шаблону вуза.

9. Статистика и аналитика. Система должна формировать сводные данные по количеству загруженных документов, числу успешных проверок, количеству найденных ошибок, активности пользователей и временным показателям обработки материалов.

10. Журналирование действий. Система должна сохранять сведения о действиях пользователей: загрузка файлов, проверка документов, вход в систему, генерация титульных листов и иные операции.

2.4 Требования к интерфейсу

Интерфейс информационной системы представляет собой веб-приложение с единым современным стилем оформления, обеспечивающее удобную и интуитивно понятную навигацию. Пользовательский интерфейс

должен быть адаптирован для работы в распространённых браузерах и обеспечивать быстрый доступ к основным функциям системы.

Основные элементы интерфейса включают:

- главную страницу системы;
- форму авторизации и регистрации пользователей;
- личный кабинет пользователя;
- страницу загрузки документов на проверку;
- страницу просмотра результатов проверки;
- страницу архива сохранённых документов;
- страницу генерации титульных листов;
- административную панель управления пользователями;
- модуль просмотра статистики и аналитических данных.

На рисунке 2.1 представлена страница авторизации системы.

Рисунок 2.1 – Страница авторизации

Как показано на рисунке, форма авторизации содержит поля ввода логина и пароля, кнопку входа в систему, а также элементы перехода к регистрации нового пользователя. После успешной авторизации пользователь получает доступ к функциональным возможностям системы в соответствии со своей ролью.

На рисунке 2.2 представлен внешний вид личного кабинета пользователя.

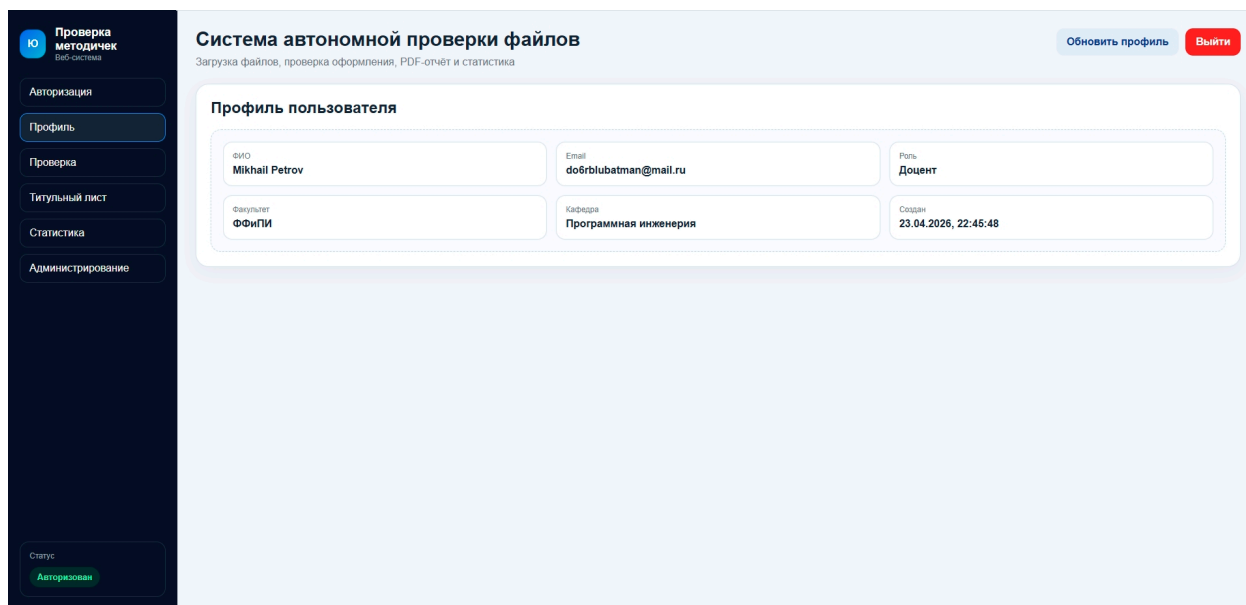


Рисунок 2.2 – Личный кабинет пользователя

Личный кабинет содержит навигационное меню, сведения о текущем пользователе, основные разделы системы, а также быстрый доступ к загрузке документов, архиву файлов и генерации титульных листов.

На рисунке 2.3 представлена страница проверки документов.

Система автономной проверки файлов
Загрузка файлов, проверка оформления, PDF-отчет и статистика

Обновить профиль Выйти

Система проверки файлов

Выберите .doc или .docx файл
Выберите файл | Файл не выбран

Проверить файл

Здесь появится результат проверки

Мои файлы Обновить список

Не удалось загрузить список методичек

Статус: Авторизован

Рисунок 2.3 – Страница проверки документов

Данная страница содержит форму выбора файла формата DOCX, кнопку запуска автоматической проверки, область отображения текущего статуса обработки и результаты анализа после завершения проверки.

На рисунке 4.1 представлена страница генерации титульных листов.

Система автономной проверки файлов
Загрузка файлов, проверка оформления, PDF-отчет и статистика

Обновить профиль Выйти

Генерация титульного листа

Тип титульного листа
Методические указания

Методические указания — первая страница

Название методички

Название дисциплины

Для кого
Студентов

Код направления
09 03 04

Название направления

Город
Курск

Год
2026

Методические указания — вторая страница

УДК

ФИО составителя

ФИО рецензента

Степень рецензента

Краткое описание
Вводите краткое описание

Не более 1000 символов 0 / 1000

Сгенерировать титульный лист

Рисунок 2.4 – Страница генерации титульных листов

Как показано на рисунке, страница содержит форму ввода реквизитов работы: наименование дисциплины, тема работы, сведения об авторе, руко-

водителе и кафедре. После заполнения формы пользователь может сформировать готовый документ в формате DOCX.

На рисунке 2.5 представлена страница статистики и аналитики.

Рисунок 2.5 – Страница статистики

Страница статистики содержит сведения о количестве зарегистрированных пользователей, числе загруженных документов, результатах проверок, количестве обнаруженных ошибок, а также иные аналитические показатели деятельности системы.

Интерфейс системы должен обеспечивать логичное расположение элементов управления, единообразие оформления страниц, наглядность представления результатов работы и минимальное количество действий пользователя при выполнении основных операций.

2.5 Требования к обработке ошибок

Система должна обеспечивать корректную обработку исключительных ситуаций.

Основные требования:

- при возникновении ошибки пользователю должно отображаться понятное сообщение;

- внутренние ошибки системы не должны раскрывать служебную информацию;
- ошибки должны фиксироваться в журнале событий;
- должна быть реализована обработка:
 - некорректных входных данных;
 - ошибок загрузки файлов;
 - ошибок обработки документов;
 - ошибок взаимодействия с базой данных.

2.6 Требования к удобству использования (UX)

Система должна обеспечивать удобство и простоту взаимодействия пользователей.

Основные требования:

- интерфейс должен быть интуитивно понятным;
- количество действий для выполнения основной операции (загрузка и проверка документа) должно быть минимальным;
- результаты проверки должны отображаться в структурированном виде;
- должна быть реализована визуальная обратная связь (индикаторы загрузки, статусы операций).

2.7 Моделирование вариантов использования

Для определения функциональных возможностей разрабатываемой системы и взаимодействия пользователей с её компонентами была выполнена модель вариантов использования.

В системе предусмотрены следующие роли пользователей:

Администратор — обладает полными правами доступа, включая управление пользователями, просмотр статистики, настройку системы, удаление документов и контроль работы пользователей.

Редактор — осуществляет просмотр загруженных материалов, контроль результатов автоматической проверки, работу с архивом документов и формирование отчётности.

Пользователь — выполняет загрузку документов, получает результаты проверки, скачивает отчёты, использует модуль генерации титульных листов и просматривает историю своих материалов.

Основные прецеденты (варианты использования) системы:

1. Вход в систему.
2. Регистрация пользователя.
3. Загрузка документа на проверку.
4. Автоматическая проверка оформления.
5. Просмотр отчёта об ошибках.
6. Скачивание исправленного документа.
7. Сохранение корректного документа в архив.
8. Поиск и просмотр ранее загруженных работ.
9. Генерация титульного листа.
10. Просмотр статистики работы системы.
11. Управление пользователями.
12. Выход из системы.

Для повышения наглядности диаграмма прецедентов была разделена по ролям пользователей. Отдельно представлены варианты использования для пользователя, администратора и редактора.

На рисунке 2.6 представлена диаграмма прецедентов пользователя.

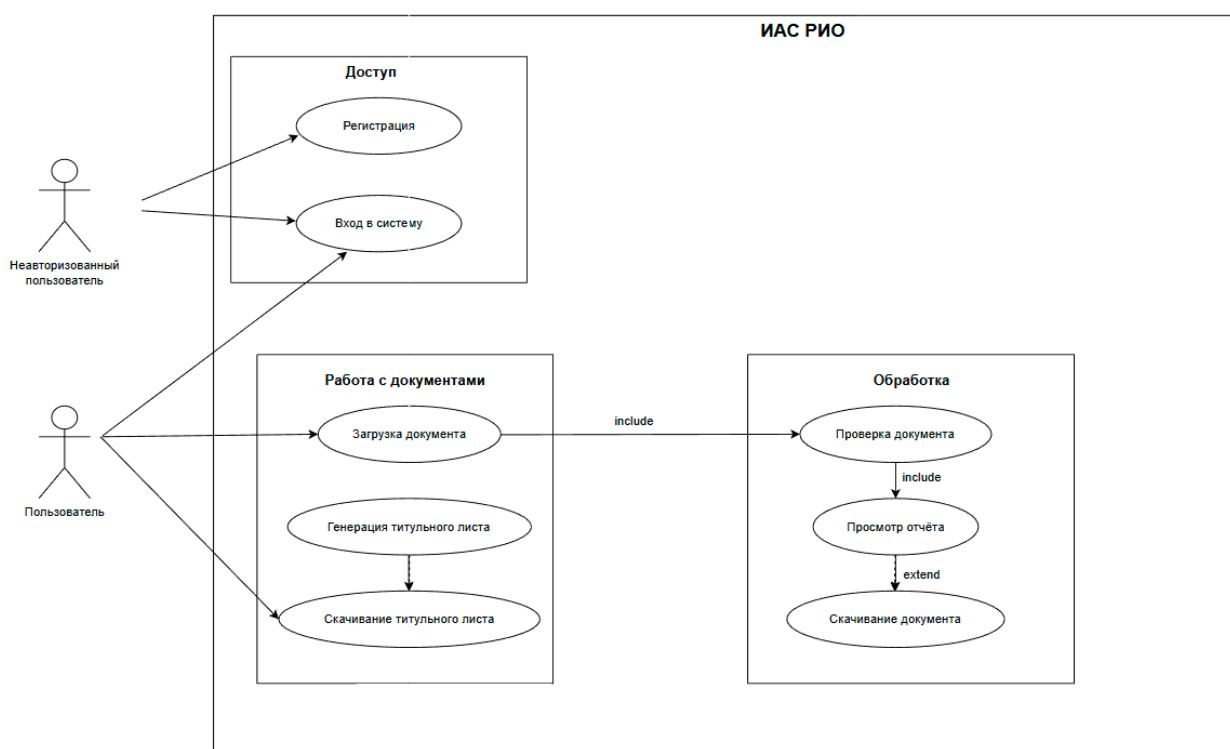


Рисунок 2.6 – Диаграмма прецедентов пользователя

На рисунке 2.7 представлена диаграмма прецедентов администратора.



Рисунок 2.7 – Диаграмма прецедентов администратора

На рисунке 2.8 представлена диаграмма прецедентов редактора.

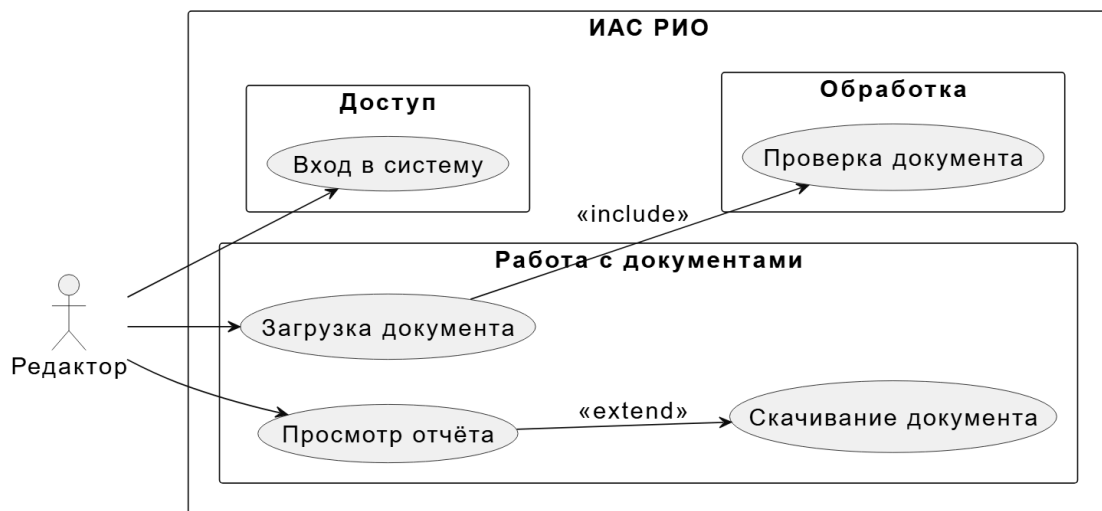


Рисунок 2.8 – Диаграмма прецедентов редактора

2.7.1 Вариант использования «Вход в систему»

Заинтересованные лица и их требования: пользователь желает получить доступ к функциям системы путём ввода логина и пароля.

Предусловие: пользователь не авторизован; открыта страница входа.

Постусловие: пользователь успешно авторизован и перенаправлен в личный кабинет либо на главную страницу системы.

Основной успешный сценарий:

1. Пользователь открывает страницу входа.
2. Вводит логин и пароль.
3. Нажимает кнопку «Войти».
4. Система выполняет поиск пользователя в базе данных.
5. Выполняется проверка пароля.
6. Создаётся пользовательская сессия.
7. Пользователь получает доступ к функциям системы.

Альтернативный сценарий (неверные данные):

1. Пользователь вводит неверный логин или пароль.
2. Система отображает сообщение об ошибке авторизации.
3. Пользователь остаётся на странице входа.

Альтернативный сценарий (пустые поля):

1. Пользователь не заполняет одно из обязательных полей.
2. Система выводит сообщение о необходимости заполнения формы.

2.7.2 Вариант использования «Выход из системы»

Заинтересованные лица и их требования: авторизованный пользователь желает завершить работу с системой.

Предусловие: пользователь авторизован.

Постусловие: пользовательская сессия завершена.

Основной успешный сценарий:

1. Пользователь нажимает кнопку «Выход».
2. Система удаляет активную сессию.
3. Пользователь перенаправляется на страницу входа.

2.7.3 Вариант использования «Загрузка документа на проверку»

Заинтересованные лица и их требования: пользователь желает отправить документ в формате DOCX для автоматической проверки.

Предусловие: пользователь авторизован.

Постусловие: файл загружен в систему и передан модулю проверки.

Основной успешный сценарий:

1. Пользователь открывает страницу загрузки.
2. Выбирает файл формата DOCX.
3. Нажимает кнопку «Загрузить».
4. Система сохраняет файл во временное хранилище.
5. Запускается процедура проверки документа.

Альтернативный сценарий (неподдерживаемый формат):

1. Пользователь выбирает файл иного формата.
2. Система выводит сообщение об ошибке.
3. Загрузка не выполняется.

Альтернативный сценарий (файл не выбран):

1. Пользователь нажимает кнопку загрузки без выбора файла.

2. Система предлагает выбрать документ.

2.7.4 Вариант использования «Автоматическая проверка документа»

Заинтересованные лица и их требования: пользователь желает получить результат проверки оформления документа.

Предусловие: документ успешно загружен.

Постусловие: сформирован результат проверки.

Основной успешный сценарий:

1. Система анализирует структуру документа.
2. Выполняется проверка параметров оформления.
3. Выявляются ошибки и несоответствия требованиям.
4. Формируется перечень найденных нарушений.

Альтернативный сценарий (ошибка обработки файла):

1. Документ повреждён либо содержит некорректную структуру.
2. Проверка прерывается.
3. Пользователь получает сообщение об ошибке обработки документа.

2.7.5 Вариант использования «Получение отчёта об ошибках»

Заинтересованные лица и их требования: пользователь желает ознакомиться с найденными нарушениями.

Предусловие: проверка завершена.

Постусловие: пользователю отображён отчёт.

Основной успешный сценарий:

1. После завершения проверки система формирует отчёт.
2. Пользователь открывает страницу результата.
3. Отображается список ошибок с пояснениями.

Альтернативный сценарий (ошибок не обнаружено):

1. Проверка завершена без замечаний.
2. Система отображает сообщение о корректности документа.

2.7.6 Вариант использования «Скачивание документа с выделенными ошибками»

Заинтересованные лица и их требования: пользователь желает скачать исправляемый файл с визуально отмеченными ошибками.

Предусловие: документ содержит нарушения.

Постусловие: пользователю предоставлен файл DOCX для скачивания.

Основной успешный сценарий:

1. Пользователь нажимает кнопку скачивания.
2. Система формирует копию документа.
3. Ошибки подчёркиваются или выделяются.
4. Пользователь скачивает файл.

Альтернативный сценарий (файл не сформирован):

1. Во время генерации файла возникает ошибка.
2. Система выводит сообщение о невозможности подготовки документа.

2.7.7 Вариант использования «Автоматическое сохранение корректного документа»

Заинтересованные лица и их требования: пользователь желает сохранить корректно оформленную работу в архиве системы.

Предусловие: ошибок в документе не обнаружено.

Постусловие: файл помещён в централизованное хранилище.

Основной успешный сценарий:

1. Проверка завершается без ошибок.
2. Система сохраняет документ в архив.
3. Информация о работе заносится в базу данных.

Альтернативный сценарий (ошибка сохранения):

1. Возникает ошибка доступа к хранилищу либо базе данных.
2. Система фиксирует ошибку в журнале событий.

3. Пользователю выводится уведомление.

2.7.8 Вариант использования «Просмотр архива документов»

Заинтересованные лица и их требования: пользователь желает просмотреть ранее сохранённые документы.

Предусловие: пользователь авторизован.

Постусловие: отображён список документов архива.

Основной успешный сценарий:

1. Пользователь открывает раздел архива.
2. Система загружает список документов.
3. Пользователь просматривает или скачивает нужный файл.

Альтернативный сценарий (архив пуст):

1. В системе отсутствуют сохранённые документы.
2. Отображается сообщение «Архив пуст».

2.7.9 Вариант использования «Формирование титульного листа»

Заинтересованные лица и их требования: пользователь желает автоматически создать титульный лист по шаблону вуза.

Предусловие: пользователь авторизован.

Постусловие: сформирован титульный лист.

Основной успешный сценарий:

1. Пользователь открывает форму генерации титульного листа.
2. Заполняет необходимые поля.
3. Нажимает кнопку формирования.
4. Система создаёт документ DOCX.

Альтернативный сценарий (не заполнены обязательные поля):

1. Пользователь оставляет обязательные поля пустыми.
2. Система сообщает о необходимости заполнения данных.

2.7.10 Вариант использования «Просмотр статистики»

Заинтересованные лица и их требования: администратор желает получить статистические сведения о работе системы.

Предусловие: администратор авторизован.

Постусловие: отображён аналитический отчёт.

Основной успешный сценарий:

1. Администратор открывает раздел статистики.
2. Система подсчитывает показатели.
3. Отображаются данные по количеству проверок, ошибок, сохранённых документов и пользователям.

Альтернативный сценарий (недостаточно прав):

1. Обычный пользователь пытается открыть раздел статистики.
2. Система запрещает доступ.

2.7.11 Вариант использования «Управление пользователями»

Заинтересованные лица и их требования: администратор желает управлять учётными записями пользователей.

Предусловие: администратор авторизован.

Постусловие: учётные записи обновлены.

Основной успешный сценарий:

1. Администратор открывает раздел пользователей.
2. Добавляет, редактирует или удаляет записи.
3. Система сохраняет изменения.

Альтернативный сценарий (дублирование логина):

1. При создании пользователя введён уже существующий логин.
2. Система выводит сообщение об ошибке.

2.7.12 Вариант использования «Настройка параметров системы»

Заинтересованные лица и их требования: администратор желает изменить правила проверки документов и параметры работы системы.

Предусловие: администратор авторизован.

Постусловие: новые параметры сохранены.

Основной успешный сценарий:

1. Администратор открывает настройки.
2. Изменяет параметры.
3. Подтверждает сохранение.
4. Система применяет новые значения.

Альтернативный сценарий (некорректные параметры):

1. Введены недопустимые значения настроек.
2. Система отменяет сохранение и выводит сообщение об ошибке.

2.8 Требования к надёжности

Разрабатываемая информационно-аналитическая система должна обеспечивать стабильную и корректную работу при различных режимах эксплуатации, а также гарантировать сохранность пользовательских данных.

Целостность данных. Все операции загрузки документов, сохранения результатов проверки, регистрации пользователей и формирования отчётов должны выполняться атомарно. При возникновении ошибки операция должна быть отменена без повреждения существующих данных.

Устойчивость к сбоям. При аварийном завершении работы сервера, отключении электропитания или программном сбое система должна сохранять ранее записанные данные. После повторного запуска система должна продолжить работу в штатном режиме.

Контроль корректности входных данных. На уровне приложения должна выполняться проверка корректности вводимых пользователями данных: обязательность заполнения полей, допустимые форматы файлов, ограничения длины строковых значений, корректность дат и иных параметров.

Логирование ошибок. Критические ошибки, исключения и события работы системы должны фиксироваться в журнале событий. Записи журнала должны содержать дату, время, уровень важности и описание ошибки.

Резервное копирование. База данных и архив документов должны допускать резервное копирование штатными средствами используемой СУБД и файловой системы. Восстановление данных должно выполняться из резервной копии.

2.9 Требования к информационной безопасности

Система должна обеспечивать защиту данных от несанкционированного доступа и разграничение прав пользователей.

Авторизация пользователей. Доступ к функциям системы должен предоставляться только после успешного ввода логина и пароля.

Хранение паролей. Пароли пользователей не должны храниться в открытом виде. Для хранения необходимо использовать криптографическое хеширование.

Разграничение прав доступа. В системе должны поддерживаться роли пользователей:

- пользователь — загрузка документов, просмотр результатов, генерация титульных листов;
- администратор — полный доступ, включая управление пользователями и просмотр статистики.

Проверка доступа на серверной стороне. Права пользователя должны проверяться сервером при выполнении каждого защищённого действия.

Защита от SQL-инъекций. Доступ к базе данных должен осуществляться с использованием ORM либо параметризованных запросов.

Защита файлового хранилища. Загруженные документы и архивные материалы должны храниться в каталогах, недоступных для прямого неавторизованного обращения.

2.10 Требования к программной и технической совместимости

Система должна функционировать в типовой вычислительной среде и не предъявлять завышенных требований к аппаратным ресурсам.

Операционная система сервера:

- Windows 10/11.

Аппаратные требования:

- процессор с тактовой частотой от 2 ГГц;
- оперативная память не менее 4 ГБ;
- свободное дисковое пространство от 1 ГБ без учёта пользовательских файлов.

Клиентская часть. Для работы пользователей необходимы текстовый редактор Word и современный веб-браузер:

- Яндекс Браузер;
- Google Chrome;
- Mozilla Firefox;
- Microsoft Edge.

2.11 Требования к оформлению документации

Разработка программной документации и программного изделия должна выполняться в соответствии с требованиями действующих нормативных документов ГОСТ 19.102–77 и ГОСТ 34.601–90. Единая система программной документации.

3 Технический проект

3.1 Архитектура программной системы

Разрабатываемая информационно-аналитическая система построена по архитектуре «клиент–сервер» с использованием REST-подхода.

Серверная часть реализована на языке программирования Python с использованием веб-фреймворка FastAPI и обеспечивает обработку HTTP-запросов, выполнение бизнес-логики, а также взаимодействие с базой данных PostgreSQL и файловым хранилищем документов.

Клиентская часть представляет собой веб-приложение, доступ к которому осуществляется через браузер. Взаимодействие между клиентом и сервером выполняется по протоколу HTTP с использованием формата JSON для обмена данными.

Архитектура системы включает следующие основные компоненты:

- клиентское веб-приложение (пользовательский интерфейс);
- серверное приложение на базе FastAPI;
- систему управления базами данных PostgreSQL;
- файловое хранилище для загружаемых и сгенерированных документов;
- модуль анализа и проверки документов формата DOCX.

Логика работы системы организована следующим образом. Пользователь через веб-интерфейс выполняет действия (авторизация, загрузка документа, запуск проверки и другие операции), которые инициируют HTTP-запросы к серверной части приложения. Сервер выполняет обработку запроса, при необходимости обращается к базе данных и файловой системе, запускает процедуры анализа документа и возвращает результат клиенту.

Модуль проверки документов осуществляет анализ структуры и параметров оформления файлов формата DOCX, выявляет несоответствия установленным требованиям и формирует отчёт о найденных ошибках. При необходимости система дополнительно создаёт копию документа с визуальным выделением нарушений.

Все загружаемые документы сохраняются в файловой системе сервера, а информация о документах (путь к файлу, пользователь, результаты проверки и другие метаданные) хранится в базе данных PostgreSQL. Такой подход обеспечивает разделение хранения файлов и структурированных данных, а также повышает масштабируемость системы.

Для обеспечения безопасности и управления доступом используется механизм аутентификации пользователей на основе JWT-токенов. После успешной авторизации пользователь получает токен доступа, который используется при выполнении последующих запросов к API системы.

Общая схема взаимодействия компонентов системы представлена на рисунке 3.1.

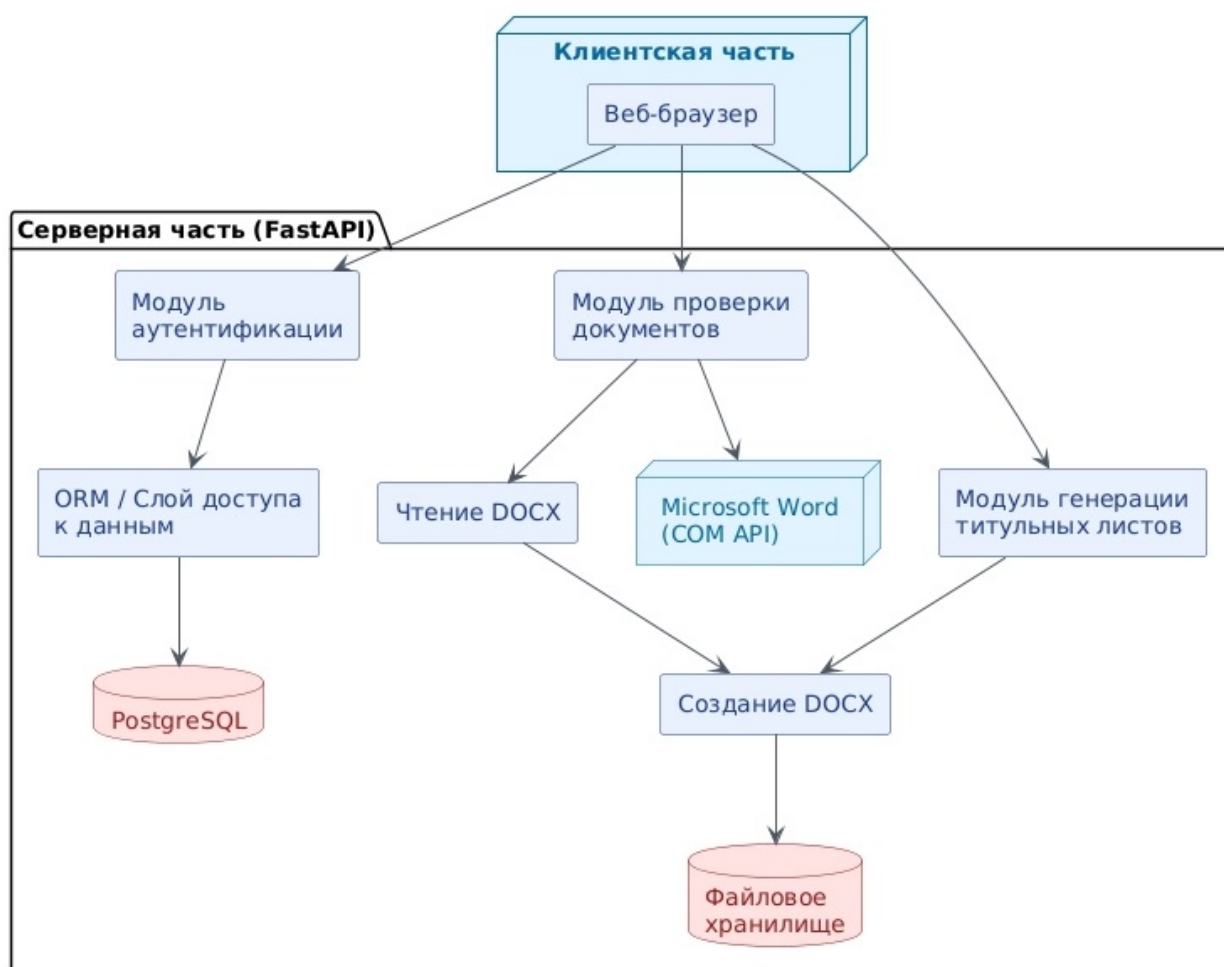


Рисунок 3.1 – Общая схема архитектуры системы

3.2 Обоснование выбора технологии проектирования

При выборе технологического стека разрабатываемой информационно-аналитической системы учитывались следующие критерии: поддержка обработки текстовых документов сложной структуры (учебников и учебных пособий), возможность расширения функциональности, удобство разработки, а также наличие инструментов для взаимодействия с документами формата DOCX и реализации веб-интерфейса.

3.2.1 Язык программирования Python

В качестве основного языка разработки выбран Python 3. Данный язык широко используется при создании информационных систем благодаря своей универсальности, высокой скорости разработки и наличию развитой экосистемы библиотек.

Выбор Python обусловлен необходимостью реализации алгоритмов автоматизированной проверки текстовых документов, а также построения серверной части веб-приложения. Язык предоставляет удобные средства для работы с файловой системой, обработки строковых данных и интеграции с внешними программными компонентами.

Для реализации функциональности системы используются следующие ключевые возможности и библиотеки Python:

1. Высокая скорость разработки и читаемость кода.
2. Наличие библиотек для работы с документами и веб-технологиями.
3. Простота интеграции с внешними системами, включая Microsoft Word.
4. Поддержка многослойной архитектуры приложения.
5. Кроссплатформенность, позволяющая развертывать систему на различных операционных системах.
6. Взаимодействие с текстовыми документами формата DOCX, включая их анализ и генерацию.

7. Интеграция с текстовым процессором Microsoft Word посредством COM-интерфейса, что позволяет получать доступ к внутренней структуре документа (абзацы, стили, форматирование).

8. Реализация веб-сервера с использованием современного фреймворка FastAPI.

9. Работа с базой данных PostgreSQL через ORM-инструменты.

10. Обработка HTTP-запросов и организация клиент-серверного взаимодействия.

Таким образом, использование Python обеспечивает эффективную реализацию алгоритмов обработки учебных и учебно-методических документов, а также удобство разработки и сопровождения информационной системы.

3.2.2 Веб-фреймворк FastAPI

В качестве основного веб-фреймворка для реализации серверной части системы выбран FastAPI — современный высокопроизводительный фреймворк на языке Python, предназначенный для разработки REST API.

Выбор FastAPI обусловлен необходимостью обработки большого количества запросов, связанных с загрузкой, анализом и проверкой текстовых документов (учебников и учебных пособий), а также формированием отчётов и генерацией документов.

Фреймворк обеспечивает асинхронную обработку HTTP-запросов, что позволяет эффективно обрабатывать несколько пользовательских операций одновременно, включая:

- загрузку документов в систему;
- запуск процедуры автоматической проверки;
- получение результатов анализа;
- скачивание обработанных файлов.

Одним из ключевых преимуществ FastAPI является встроенная поддержка валидации данных на основе аннотаций типов Python. Это позволяет автоматически проверять корректность входных данных (например, пара-

метров запроса или метаданных документа), снижая вероятность ошибок на этапе обработки.

Механизм маршрутизации FastAPI связывает URL-адреса с функциями-обработчиками (endpoint). Например:

1. Запрос на загрузку документа направляется к обработчику загрузки файла.
2. Запрос на запуск проверки — к модулю анализа документа.
3. Запрос на получение результата — к обработчику формирования отчёта.

Дополнительным преимуществом является автоматическая генерация интерактивной документации, что упрощает тестирование и отладку системы.

Таким образом, использование FastAPI позволяет реализовать масштабируемую и производительную серверную часть системы, обеспечивающую обработку учебных и учебно-методических документов в многопользовательском режиме.

3.2.3 СУБД PostgreSQL

В качестве системы управления базами данных выбрана PostgreSQL — современная объектно-реляционная СУБД, обеспечивающая высокую надёжность, производительность и поддержку сложных структур данных.

Выбор PostgreSQL обусловлен необходимостью хранения и обработки различных типов данных, используемых в информационно-аналитической системе редакционно-издательского отдела. К ним относятся:

- учетные записи пользователей и их роли;
- информация о загруженных документах;
- результаты автоматизированной проверки;
- отчёты об ошибках и история обработки документов;
- служебные данные системы.

PostgreSQL обеспечивает поддержку ссылочной целостности данных за счёт использования внешних ключей и каскадных операций. Это позволя-

ет автоматически поддерживать согласованность данных при удалении связанных сущностей, таких как документы и результаты их проверки.

Использование транзакций гарантирует целостность данных при выполнении операций добавления, изменения и удаления. В случае возникновения ошибок возможен откат изменений, что исключает появление неконсистентных данных.

Для взаимодействия с базой данных применяется ORM-технология, позволяющая работать с данными на уровне объектов приложения. Это упрощает разработку, повышает читаемость кода и снижает вероятность ошибок при формировании SQL-запросов.

Сервер базы данных функционирует как отдельный компонент системы и взаимодействует с серверной частью приложения по сети. Такой подход обеспечивает:

- возможность масштабирования системы;
- одновременную работу нескольких пользователей;
- централизованное хранение данных;
- повышение отказоустойчивости.

Для повышения производительности используется механизм пула соединений с базой данных, позволяющий переиспользовать уже установленные подключения. Это снижает накладные расходы на установление соединений и обеспечивает эффективную обработку параллельных запросов.

Таким образом, PostgreSQL обеспечивает надёжное хранение данных и эффективную работу системы при обработке учебных и учебно-методических документов.

3.2.4 Средства работы с документами

Ключевым аспектом разрабатываемой системы является обработка текстовых документов формата DOCX, содержащих учебники и учебные пособия. Для реализации данной функциональности используются специализированные средства работы с документами Microsoft Word.

Основной механизм анализа документов реализован с использованием COM-интерфейса Microsoft Word. Данный подход позволяет работать с документом на уровне его внутренней структуры, включая:

- доступ к абзацам, разделам и стилям оформления;
- получение параметров форматирования (шрифт, размер, интервалы, отступы);
- определение структуры документа (заголовки, списки, таблицы);
- модификацию документа (выделение ошибок, добавление комментариев).

Использование COM-интерфейса обеспечивает более точный и полный анализ документов по сравнению с альтернативными библиотеками, так как позволяет учитывать особенности форматирования, применяемые в текстовом процессоре Microsoft Word.

Для генерации документов (например, титульных листов) используется библиотека `python-docx`. Она позволяет программно создавать DOCX-файлы, задавать структуру документа и управлять параметрами форматирования.

Таким образом, совокупное использование COM-интерфейса и библиотеки `python-docx` обеспечивает как анализ, так и генерацию учебных и учебно-методических документов.

3.2.5 Прочие библиотеки и инструменты

В процессе разработки информационно-аналитической системы используются дополнительные библиотеки и инструменты, обеспечивающие реализацию вспомогательной функциональности.

Для организации аутентификации и авторизации пользователей применяются механизмы на основе JWT (JSON Web Token), обеспечивающие безопасную передачу данных между клиентом и сервером.

Логирование работы системы реализовано с использованием стандартного модуля `Python logging`. В журнал событий записываются:

- действия пользователей (загрузка документов, запуск проверки);
- ошибки и исключительные ситуации;

– служебные сообщения системы.

Это позволяет выполнять мониторинг работы системы и упрощает процесс отладки.

Для запуска серверной части используется ASGI-сервер Uvicorn, обеспечивающий высокую производительность и поддержку асинхронной обработки запросов.

Дополнительно применяются средства работы с файловой системой Python для хранения загруженных документов, формирования временных файлов и организации архива обработанных материалов.

Таким образом, используемые библиотеки и инструменты обеспечивают стабильную и безопасную работу системы, а также поддержку основных процессов обработки учебных и учебно-методических документов.

3.3 Описание серверной части

Серверная часть информационно-аналитической системы является основным компонентом, реализующим бизнес-логику обработки учебных и учебно-методических изданий. Именно на серверной стороне выполняются операции анализа документов, проверки параметров форматирования, генерации титульных листов, формирования отчётов и взаимодействия с базой данных.

Архитектура серверной части построена на основе веб-фреймворка FastAPI, обеспечивающего обработку HTTP-запросов, маршрутизацию, взаимодействие между компонентами системы и предоставление REST API для клиентской части приложения.

Взаимодействие клиента и сервера осуществляется по протоколу HTTP в формате запрос–ответ. Пользователь через веб-интерфейс отправляет запросы на загрузку документов, запуск проверки, генерацию титульных листов и получение результатов обработки. Сервер принимает запросы, выполняет необходимую обработку данных и возвращает результаты пользователю.

Серверная часть реализует REST API, предназначенный для:

- аутентификации и авторизации пользователей;
- загрузки документов формата DOCX;
- запуска процедуры автоматизированной проверки;
- получения результатов анализа;
- генерации титульных листов;
- скачивания сформированных отчётов и обработанных документов.

Ключевым компонентом серверной части является модуль обработки документов, реализующий алгоритмы анализа учебников и учебных пособий. Проверка документов осуществляется посредством взаимодействия с текстовым процессором Microsoft Word через COM-интерфейс, что позволяет получать доступ к внутренней структуре DOCX-документа:

- абзацам;
- разделам;
- стилям оформления;
- параметрам страницы;
- таблицам и спискам;
- элементам форматирования текста.

В процессе анализа выполняется последовательная проверка параметров форматирования документа на соответствие установленным требованиям. Результатом обработки является структурированный отчёт, содержащий сведения о выявленных ошибках, их типах и расположении в документе.

Дополнительно серверная часть реализует механизм визуальной разметки документов. На основе результатов проверки формируется копия исходного файла, содержащая выделение ошибок и комментарии, упрощающие процесс исправления нарушений оформления.

Отдельным компонентом серверной части является модуль генерации титульных листов. Генерация осуществляется на основе шаблонного подхода: пользователь вводит необходимые параметры через веб-интерфейс, после чего сервер формирует готовый DOCX-документ с автоматически заполненными страницами титульной части учебного издания.

Для хранения данных используется система управления базами данных PostgreSQL. В базе данных сохраняются:

- данные пользователей;
- сведения о кафедрах и факультетах;
- информация о загруженных документах;
- результаты проверок;
- данные аутентификации и refresh-токены;
- служебная информация системы.

Взаимодействие с базой данных реализовано с использованием ORM SQLAlchemy, обеспечивающей объектно-реляционное отображение данных и упрощающей выполнение операций чтения, записи и обновления информации.

Для обеспечения безопасности доступа к системе используется механизм JWT-аутентификации (JSON Web Token). После успешной авторизации сервер формирует access- и refresh-токены, обеспечивающие безопасное взаимодействие клиента с защищёнными ресурсами системы.

Серверная часть включает несколько взаимосвязанных модулей:

- модуль маршрутизации и обработки HTTP-запросов;
- модуль аутентификации и авторизации;
- модуль проверки учебных изданий;
- модуль генерации титульных листов;
- модуль формирования отчётов;
- модуль взаимодействия с базой данных.

Взаимодействие модулей построено по принципу разделения ответственности, при котором каждый компонент выполняет строго определённые функции. Такой подход обеспечивает модульность архитектуры, упрощает сопровождение системы и позволяет расширять функциональность без изменения существующих компонентов.

Общая схема взаимодействия компонентов серверной части представлена на рисунке 3.2.

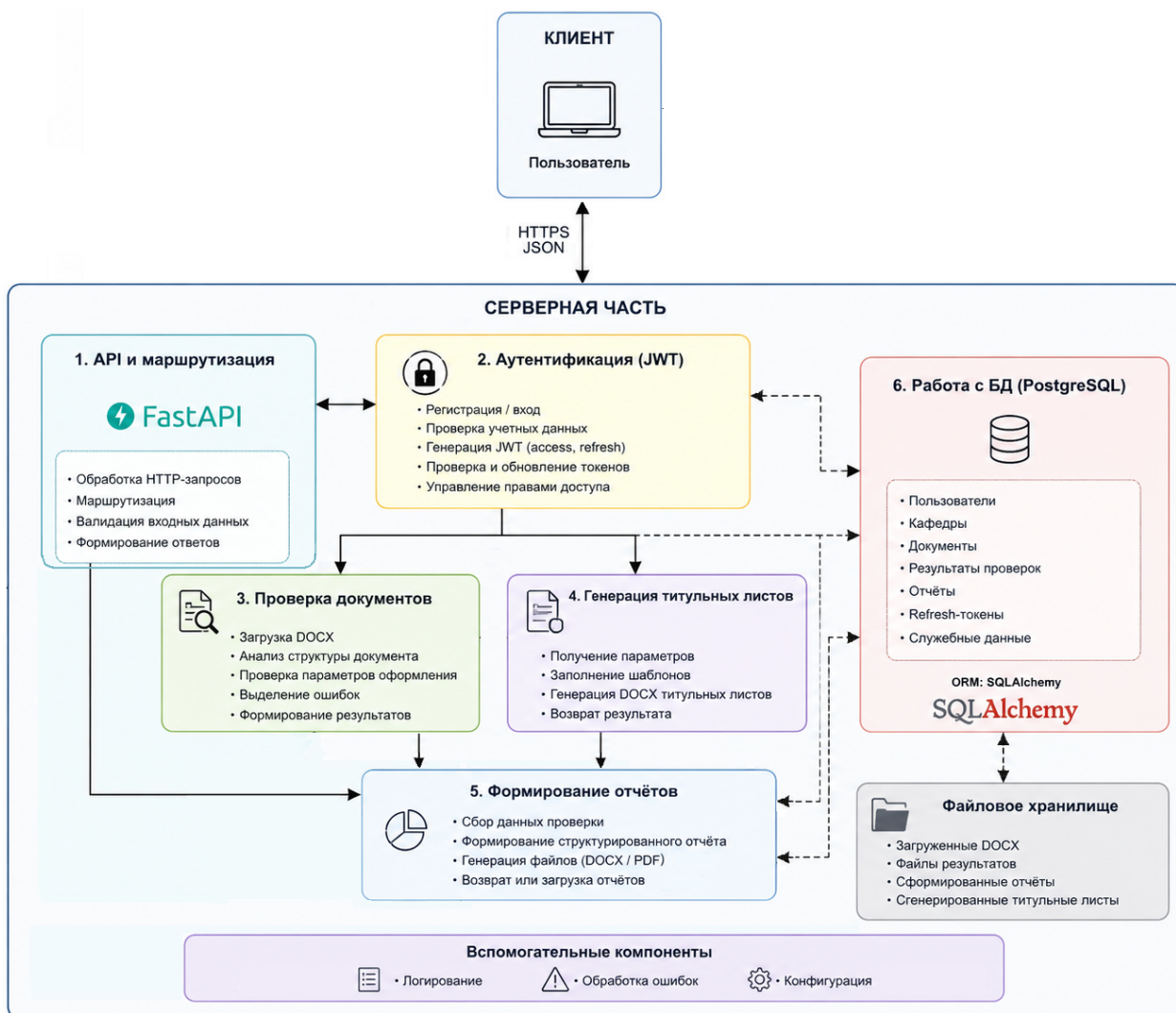


Рисунок 3.2 – Общая схема взаимодействия компонентов серверной части

3.4 Описание API

Взаимодействие клиентской части с сервером реализовано посредством REST API, построенного на основе веб-фреймворка FastAPI. API обеспечивает обмен данными между пользовательским интерфейсом и внутренними компонентами системы, а также предоставляет набор endpoint-обработчиков для выполнения основных операций.

Обмен данными осуществляется по протоколу HTTP. Для передачи структурированных данных используется формат JSON, а загрузка документов выполняется с использованием формата multipart/form-data.

Основными функциями API являются:

- обработка HTTP-запросов;

- маршрутизация запросов между компонентами системы;
- валидация входных данных;
- формирование HTTP-ответов;
- обработка ошибок и исключений;
- обеспечение взаимодействия клиентской части с серверной логикой.

Архитектура API построена по модульному принципу. Для разделения функциональности используются отдельные маршрутизаторы (routers), отвечающие за конкретные подсистемы приложения.

Общая схема взаимодействия маршрутизаторов серверной части представлена на рисунке 3.3.

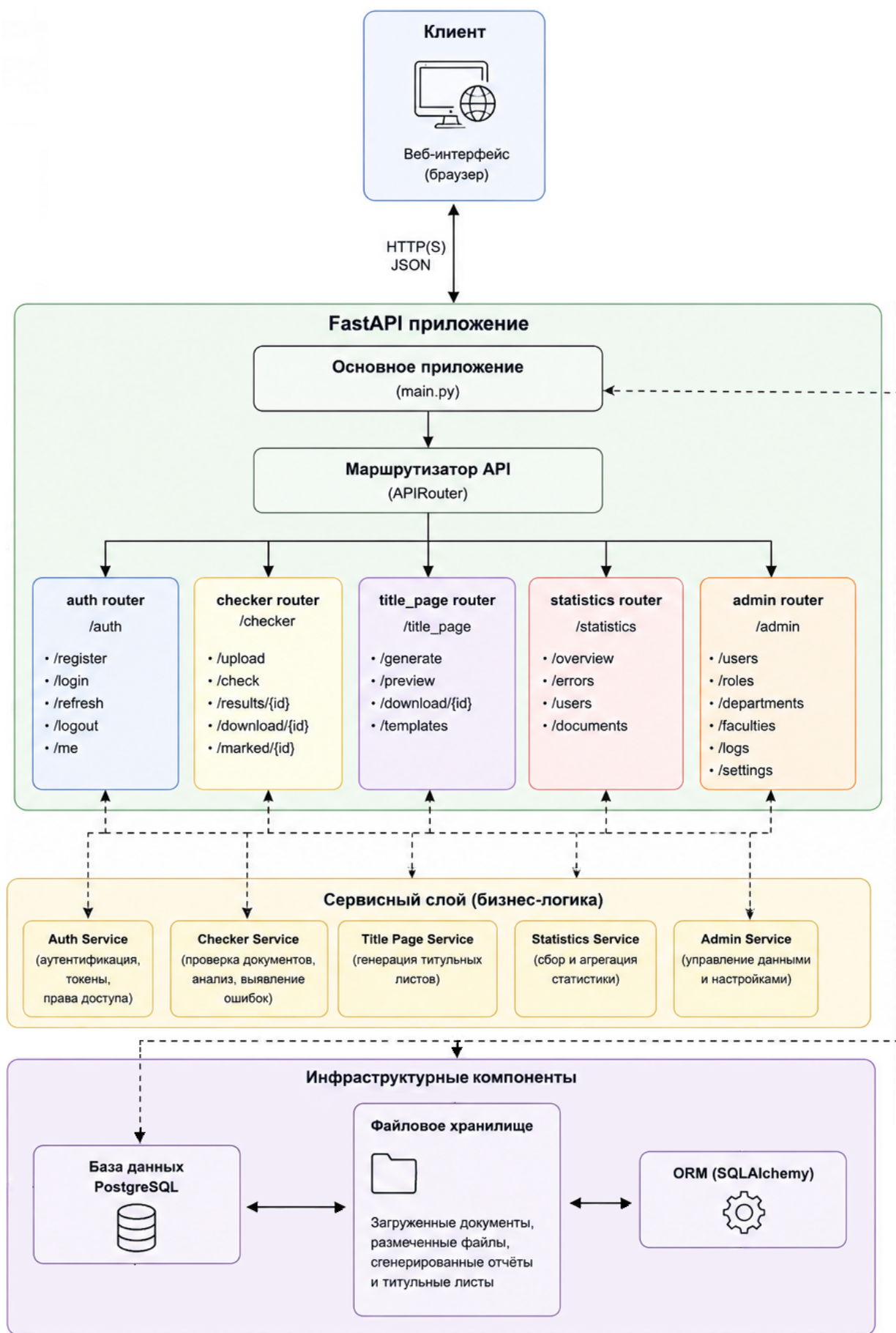


Рисунок 3.3 – Структура маршрутизации серверной части

В системе реализованы следующие основные маршрутизаторы:

1. auth — модуль аутентификации и авторизации пользователей.
2. checker — модуль загрузки и проверки документов.
3. title_page — модуль генерации титульных листов.
4. statistics — модуль получения статистических данных.
5. admin — административный модуль управления системой.

Маршрутизатор auth обеспечивает:

- регистрацию пользователей;
- вход в систему;
- обновление access-токенов;
- завершение пользовательской сессии;
- получение информации о текущем пользователе.

Маршрутизатор checker предназначен для:

- загрузки документов формата DOCX;
- запуска процедуры проверки;
- получения результатов анализа;
- скачивания отчётов;
- получения размеченных документов.

Маршрутизатор title_page реализует:

- приём параметров генерации;
- формирование DOCX-документов;
- выдачу готовых титульных листов пользователю.

Маршрутизатор statistics обеспечивает получение:

- статистики проверок;
- сведений об ошибках;
- данных об активности пользователей.

Маршрутизатор admin предоставляет административные функции:

- управление пользователями;
- изменение ролей;
- просмотр журналов действий;
- управление справочными данными системы.

FastAPI автоматически выполняет:

- сопоставление URL-адресов с endpoint-функциями;
- проверку типов входных данных;
- сериализацию и десериализацию объектов;
- формирование JSON-ответов;
- обработку ошибок валидации.

При поступлении запроса сервер выполняет маршрутизацию и передаёт управление соответствующему endpoint-обработчику. Далее запрос направляется в сервисный модуль системы, реализующий необходимую бизнес-логику. После завершения обработки формируется HTTP-ответ, возвращаемый клиентской части.

Дополнительно API реализует централизованную обработку исключений. В случае возникновения ошибок сервер формирует структурированный ответ, содержащий код ошибки и описание проблемы.

Использование REST API обеспечивает модульность, масштабируемость и возможность дальнейшего расширения функциональности системы.

3.4.1 Примеры JSON-запросов

Пример JSON-запроса авторизации пользователя

POST /auth/login

```
{  
  "email": "admin@example.com",  
  "password": "password123"  
}
```

Пример JSON-ответа при успешной авторизации

```
{  
  "access_token": "eyJhbGciOiJIUzI1NiIs...",  
  "refresh_token": "eyJhbGciOiJIUzI1NiIs...",  
  "token_type": "bearer",  
  "role": "admin"
```



```
}
```

Пример JSON-запроса на обновление access-токена

POST /auth/refresh

```
{  
  "refresh_token": "eyJhbGciOiJIUzI1NiIs..."  
}
```

Пример JSON-ответа при обновлении токена

```
{  
  "access_token": "eyJhbGciOiJIUzI1NiIs...",  
  "token_type": "bearer"  
}
```

Пример JSON-ответа с информацией о пользователе

```
{  
  "id": 15,  
  "fio": "Иванов Иван Иванович",  
  "email": "ivanov@example.com",  
  "role": "teacher",  
  "department": "Кафедра программной инженерии"  
}
```

Пример JSON-ответа с результатами проверки документа

```
{  
  "status": "completed",  
  "document": "metodichka.docx",  
  "errors_count": 3,  
  "errors": [  
    {  
      "type": "font_size",  
      "page": 5,  
      "paragraph": 18,  
      "description": "Неверный размер шрифта"    }  
  ]  
}
```

```
},  
{  
  "type": "line_spacing",  
  "page": 7,  
  "paragraph": 25,  
  "description": "Некорректный межстрочный интервал"  
}  
]  
}
```

Пример JSON-запроса генерации титульного листа

POST /title_page/generate

```
{  
  "title": "Разработка информационной системы",  
  "author": "Иванов И.И.",  
  "faculty": "Факультет информатики",  
  "department": "Кафедра программной инженерии",  
  "year": 2025  
}
```

Пример JSON-ответа сервера при генерации титульного листа

```
{  
  "status": "success",  
  "file": "title_page.docx",  
  "download_url": "/downloads/title_page.docx"  
}
```

Пример JSON-ответа при возникновении ошибки

```
{  
  "detail": "Документ имеет неподдерживаемый формат"  
}
```

3.5 Модуль маршрутизации и обработки HTTP-запросов

Модуль маршрутизации и обработки HTTP-запросов является центральным компонентом серверной части информационно-аналитической системы. Данный модуль обеспечивает взаимодействие между клиентской частью приложения и внутренними сервисами серверной части.

Реализация модуля основана на веб-фреймворке FastAPI, обеспечивающем обработку HTTP-запросов, маршрутизацию и вызов соответствующих обработчиков.

Основными задачами модуля являются:

- приём HTTP-запросов;
- распределение запросов между серверными компонентами;
- вызов сервисных модулей;
- управление потоком обработки запросов;
- формирование HTTP-ответов.

Архитектура маршрутизации построена по модульному принципу. Для разделения логики используются отдельные маршрутизаторы (routers), связанные с функциональными подсистемами приложения.

Каждый маршрутизатор содержит набор endpoint-обработчиков, реализующих отдельные операции системы.

Модуль маршрутизации взаимодействует:

- с модулем аутентификации;
- с модулем проверки документов;
- с модулем генерации титульных листов;
- с модулем формирования отчётов;
- с модулем работы с базой данных.

После получения HTTP-запроса FastAPI выполняет маршрутизацию запроса и передаёт управление соответствующему обработчику. Далее запрос направляется в сервисный слой системы, реализующий бизнес-логику.

Результатом работы модуля является сформированный HTTP-ответ, возвращаемый клиентской части приложения.

Использование модульной системы маршрутизации позволяет:

- разделить ответственность между компонентами;
- упростить сопровождение серверной части;
- обеспечить масштабируемость приложения;
- упростить расширение функциональности системы.

3.6 Модуль аутентификации и авторизации

Модуль аутентификации и авторизации предназначен для обеспечения безопасного доступа пользователей к функциям информационно-аналитической системы. Данный модуль реализует механизмы идентификации пользователей, проверки прав доступа и управления пользовательскими сессиями.

В разработанной системе аутентификация реализована на основе технологии JWT (JSON Web Token). Использование токен-ориентированного подхода позволяет отказаться от хранения пользовательских сессий на сервере и обеспечивает безопасное взаимодействие между клиентской и серверной частями приложения.

Модуль аутентификации включает следующие основные компоненты:

1. `auth.py` — содержит маршруты авторизации и обновления токенов.
2. `auth_service.py` — реализует бизнес-логику аутентификации пользователей.
3. `refresh_token.py` — отвечает за создание, хранение и отзыв `refresh`-токенов.
4. `security.py` — реализует функции генерации JWT-токенов, хеширования и проверки паролей.
5. `dependencies.py` — содержит зависимости FastAPI для проверки авторизации и прав доступа.

3.6.1 Процесс аутентификации пользователя

Процесс входа пользователя в систему включает следующие этапы:

1. Пользователь вводит логин и пароль через веб-интерфейс.

2. Клиентская часть отправляет HTTP-запрос на серверный маршрут авторизации.

3. Модуль `auth.py` принимает запрос и передаёт данные в сервис аутентификации.

4. Компонент `auth_service.py` выполняет:

- поиск пользователя в базе данных;
- проверку корректности пароля;
- определение роли пользователя.

5. При успешной проверке создаются:

- access token;
- refresh token.

6. Access token возвращается клиенту для последующего доступа к API.

7. Refresh token сохраняется в базе данных и используется для обновления access token без повторного ввода пароля.

На рисунке 3.4 представлена диаграмма последовательности процесса аутентификации пользователя.

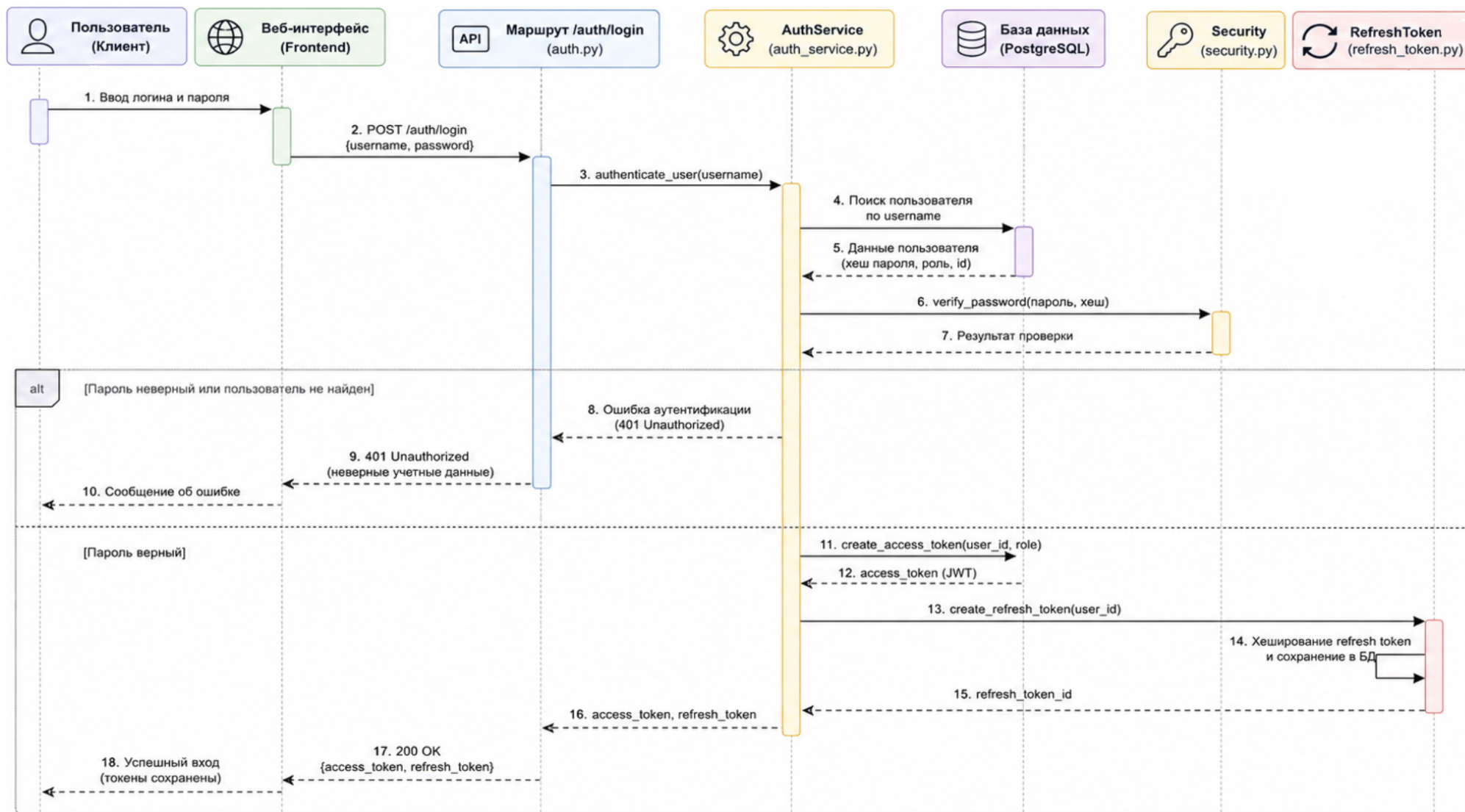


Рисунок 3.4 – Диаграмма последовательности процесса аутентификации

3.6.2 JWT-токены

Для реализации механизма авторизации используются два типа токенов:

- **Access token** — короткоживущий токен, используемый для доступа к защищённым маршрутам API;
- **Refresh token** — долгоживущий токен, предназначенный для обновления access token.

JWT-токены содержат зашифрованную информацию о пользователе:

- идентификатор пользователя;
- роль пользователя;
- время создания токена;
- срок действия токена.

Генерация и проверка JWT выполняется модулем security.py. Для повышения безопасности токены подписываются секретным ключом с использованием криптографического алгоритма.

3.6.3 Хеширование паролей

Пароли пользователей не хранятся в открытом виде. Перед сохранением в базе данных выполняется их хеширование с использованием криптографических функций.

Модуль security.py реализует:

- генерацию хеша пароля;
- проверку соответствия введённого пароля сохранённому хешу.

Данный подход обеспечивает защиту пользовательских данных даже при компрометации базы данных.

3.6.4 Роли пользователей и разграничение прав доступа

В системе реализована ролевая модель доступа. Каждому пользователю назначается определённая роль, определяющая набор доступных функций.

В системе предусмотрены следующие основные роли:

1. Администратор.
2. Сотрудник редакционно-издательского отдела.
3. Обычный пользователь.

Проверка прав доступа выполняется с использованием зависимостей FastAPI, реализованных в модуле `dependencies.py`.

При обращении к защищённому маршруту:

1. Выполняется проверка наличия `access token`.
2. Производится декодирование JWT.
3. Определяется роль пользователя.
4. Проверяется наличие необходимых прав доступа.

При отсутствии необходимых прав сервер возвращает сообщение об ошибке авторизации.

3.6.5 Хранение refresh token

Refresh token сохраняются в таблице `RefreshToken` базы данных. Для каждого токена хранятся:

- идентификатор пользователя;
- хеш токена;
- дата создания;
- срок действия;
- информация об устройстве и IP-адресе;
- признак отзыва токена.

Использование отдельной таблицы `refresh token` позволяет:

- завершать пользовательские сессии;
- отзывать токены при выходе пользователя;
- контролировать активные подключения;
- повышать безопасность системы.

3.6.6 Dependencies и middleware

Для защиты маршрутов FastAPI используются специальные зависимости (dependencies), автоматически выполняющие проверку авторизации перед вызовом обработчика запроса.

Dependencies обеспечивают:

- получение текущего пользователя;
- проверку JWT-токена;
- проверку роли пользователя;
- ограничение доступа к административным маршрутам.

Дополнительно в системе могут использоваться middleware-компоненты, выполняющие:

- логирование запросов;
- контроль времени обработки;
- обработку ошибок авторизации;
- централизованную обработку исключений.

Таким образом, модуль аутентификации и авторизации обеспечивает безопасный доступ пользователей к функциональности системы, реализует разграничение прав доступа и поддерживает механизм защищённых пользовательских сессий.

3.7 Модуль проверки документов

Одним из ключевых компонентов серверной части информационно-аналитической системы является модуль проверки учебных изданий, реализующий автоматизированный анализ учебников и учебных пособий на соответствие установленным требованиям оформления.

Основная логика проверки реализована в модуле checker.py. Данный модуль выполняет структурный анализ документов формата DOCX, извлечение параметров форматирования и формирование результатов проверки.

Основной задачей модуля является автоматизация процесса контроля оформления учебных изданий, что позволяет существенно сократить время

проверки документов и уменьшить количество ошибок, связанных с человеческим фактором.

3.7.1 Основные функции модуля проверки

Модуль проверки обеспечивает выполнение следующих функций:

1. Загрузка и открытие документа формата DOCX.
2. Анализ структуры документа.
3. Последовательный обход абзацев и элементов документа.
4. Проверка параметров форматирования.
5. Выявление и классификация ошибок.
6. Формирование отчёта о проверке.
7. Создание копии документа с визуальной разметкой ошибок.

3.8 Модуль генерации титульных листов

Одним из компонентов серверной части информационно-аналитической системы является модуль генерации титульных листов, предназначенный для автоматического формирования титульной части документов в соответствии с установленными требованиями оформления.

Основная логика реализована в следующих компонентах:

1. title_page.py — обработка API-запросов.
2. title_page_generator.py — генерация DOCX-документов.
3. schemas/title_page.py — валидация входных данных.

3.8.1 Назначение модуля

Модуль предназначен для:

1. Автоматизации подготовки титульных листов.
2. Исключения ошибок ручного форматирования.
3. Обеспечения единообразного оформления документов.
4. Сокращения времени подготовки документации.

3.8.2 Генерация DOCX-документов

В основе работы модуля используется шаблонный подход. После проверки пользовательских данных система автоматически формирует DOCX-документ с применением установленных параметров форматирования.

В процессе генерации:

- создаётся структура документа;
- выполняется размещение текстовых блоков;
- применяются параметры форматирования;
- сохраняется итоговый DOCX-файл.

3.8.3 Форматирование титульного листа

При генерации документа автоматически контролируются:

- параметры шрифта;
- межстрочные интервалы;
- выравнивание текста;
- размеры полей страницы;
- расположение текстовых блоков.

Таким образом, модуль обеспечивает автоматизированное создание титульных листов и снижает вероятность ошибок оформления.

3.9 Модуль формирования отчётов

Модуль формирования отчётов предназначен для автоматического создания PDF-документов по результатам проверки учебных изданий. Основная логика реализована в файле `pdf_report.py`.

Модуль получает результаты анализа документа, выполняет обработку выявленных ошибок и формирует структурированный PDF-отчёт.

3.9.1 Основные функции модуля

Основные функции модуля:

- формирование PDF-отчётов;

- группировка ошибок по типам;
- сортировка нарушений;
- отображение статистики проверки;
- визуализация результатов анализа.

При генерации используются:

- сведения о документе;
- дата и время проверки;
- список обнаруженных ошибок;
- информация о страницах и абзацах с нарушениями.

3.9.2 Группировка ошибок

Для уменьшения объёма отчёта реализован механизм агрегации ошибок. Однотипные нарушения объединяются в диапазоны и отображаются как единая запись.

Это позволяет:

- сократить объём отчёта;
- упростить анализ результатов проверки;
- повысить читаемость документа.

3.9.3 Структура PDF-отчёта

Итоговый отчёт включает:

1. Информацию о документе.
2. Статистику проверки.
3. Список выявленных ошибок.
4. Итоговые результаты анализа.

В процессе генерации выполняется оформление заголовков, таблиц, разделов и нумерации страниц.

3.9.4 Взаимодействие с компонентами системы

Модуль взаимодействует со следующими компонентами:

- модулем проверки документов;

- базой данных;
- REST API;
- модулем визуальной разметки.

Сформированный PDF-документ сохраняется на сервере и становится доступным пользователю через веб-интерфейс системы.

3.10 Модуль работы с базой данных

Модуль работы с базой данных обеспечивает хранение и обработку информации информационно-аналитической системы. В качестве СУБД используется PostgreSQL, а взаимодействие с базой данных реализовано с использованием ORM SQLAlchemy.

Основные задачи модуля:

1. Хранение данных пользователей.
2. Хранение информации о документах.
3. Сохранение результатов проверки.
4. Управление токенами аутентификации.
5. Обеспечение целостности данных.

Модуль включает:

- каталог `models/` с ORM-моделями;
- каталог `db/` для подключения к базе данных;
- механизм ORM SQLAlchemy.

Взаимодействие с базой данных осуществляется через ORM-сессии SQLAlchemy, обеспечивающие:

- добавление данных;
- выборку информации;
- обновление записей;
- удаление сущностей;
- фиксацию транзакций.

На рисунке 3.5 представлена схема взаимодействия программной системы с базой данных.

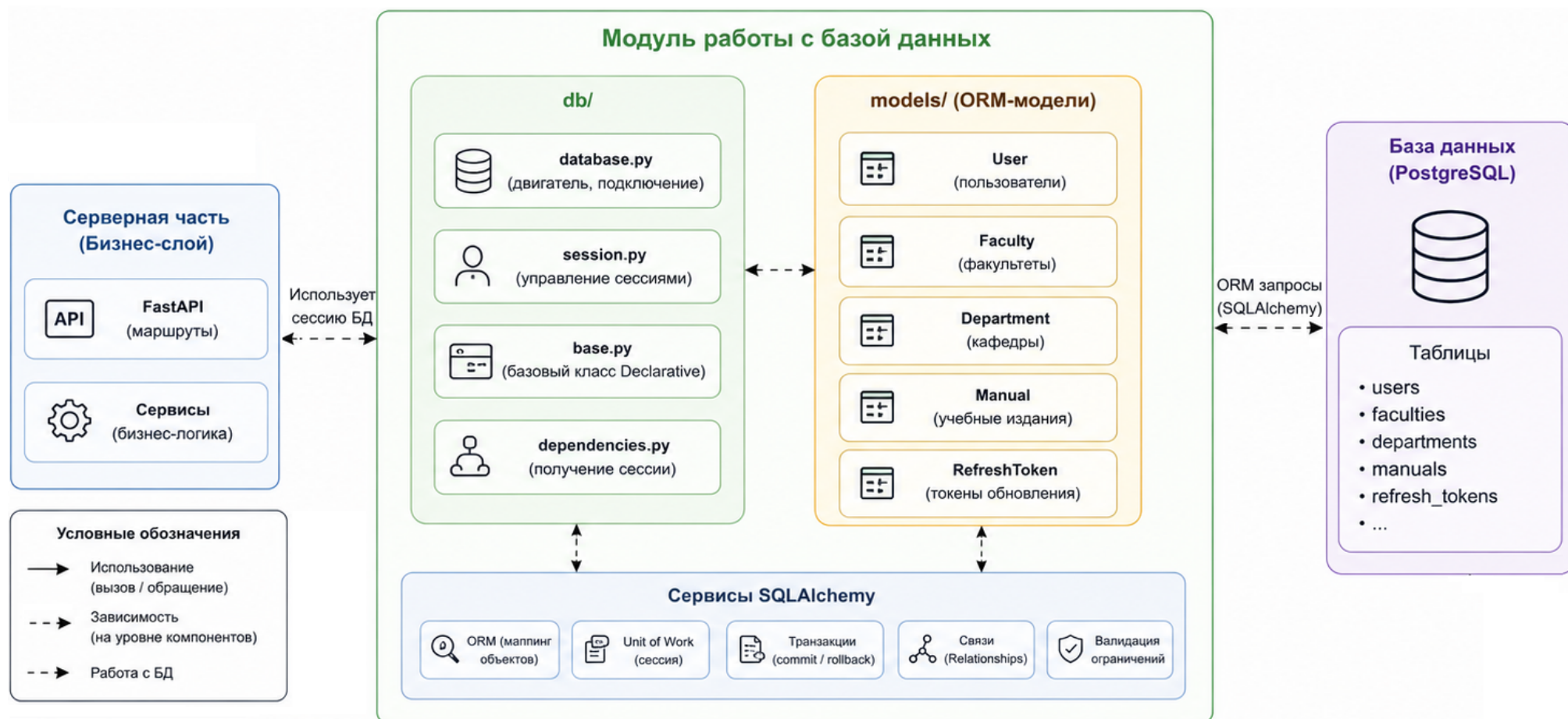


Рисунок 3.5 – Схема взаимодействия программной системы с базой данных

3.11 Форматы данных

Обмен данными между клиентской и серверной частями информационно-аналитической системы осуществляется по протоколу HTTP в формате запрос–ответ. Пользователь через веб-интерфейс выполняет загрузку учебных изданий, запуск проверки документов, генерацию титульных листов и получение результатов обработки.

Взаимодействие с REST API серверной части реализовано с использованием формата JSON. Данный формат применяется при передаче служебной информации, параметров запросов, результатов проверки и ответов API. Использование JSON обеспечивает универсальность обмена данными и упрощает взаимодействие клиентской и серверной частей системы.

При загрузке документов используется формат multipart/form-data, обеспечивающий передачу бинарных файлов. Данный механизм применяется при отправке учебников и учебных пособий в формате DOCX на сервер для последующего анализа.

Основным форматом обрабатываемых документов является **DOCX**. Формат DOCX используется для представления учебников и учебных пособий и позволяет выполнять программный доступ к структуре документа, включая:

- абзацы;
- стили оформления;
- разделы;
- таблицы;
- параметры страниц;
- элементы форматирования.

Результаты обработки документов представляются в нескольких форматах:

1. JSON-структуры, содержащие сведения о результатах проверки, найденных ошибках и служебной информации.

2. PDF-отчёты, формируемые модулем генерации отчётов и содержащие структурированную информацию о выявленных нарушениях оформления.

3. DOCX-файлы с визуальной разметкой ошибок и комментариями.

4. Сгенерированные титульные листы учебников и учебных пособий в формате DOCX.

Взаимодействие с базой данных осуществляется посредством ORM SQLAlchemy. Данные сохраняются в реляционном виде в таблицах пользователей, факультетов, кафедр, учебных материалов и токенов обновления.

Файлы учебных изданий хранятся в файловой системе сервера, а в базе данных сохраняются их метаданные, включая:

- название документа;
- путь к файлу;
- дату загрузки;
- сведения о пользователе;
- статус обработки.

Журналирование работы системы выполняется в текстовом формате с использованием временных меток и уровней событий (INFO, WARNING, ERROR). Журнал содержит сведения о действиях пользователей, этапах обработки документов и возникающих ошибках, что позволяет выполнять мониторинг работы системы и упрощает процесс отладки.

Таким образом, используемые форматы данных обеспечивают корректную передачу, обработку и хранение информации в рамках информационно-аналитической системы автоматизированной проверки учебников и учебных пособий.

4 Рабочий проект

4.1 Спецификация классов программной системы

В данном разделе приведено описание основных классов и модулей информационно-аналитической системы, реализованных в процессе разработки серверной и клиентской части приложения. Для каждого класса указываются его назначение, основные методы и функции, а также описание выполняемых операций.

4.1.1 Класс User (модель пользователя)

Класс User (модуль `models/user.py`) предназначен для представления учётной записи пользователя в системе. Модель реализована с использованием ORM SQLAlchemy и содержит сведения о пользователе: ФИО, адрес электронной почты, пароль, роль, факультет и кафедру. Класс используется при аутентификации пользователей, разграничении прав доступа и формировании статистики.

Описание методов класса User представлено в таблице 4.1.

Таблица 4.1 – Методы класса User (`models/user.py`)

Метод	Доступ	Описание
<code>__init__(self, email, password, role)</code>	public	Конструктор объекта пользователя
<code>verify_password(self, password)</code>	public	Проверяет корректность пароля пользователя
<code>create_tokens(self)</code>	public	Генерирует JWT-токены пользователя
<code>to_dict(self)</code>	public	Преобразует объект пользователя в словарь

4.1.2 Класс Faculty (модель факультета)

Класс Faculty (модуль models/faculty.py) предназначен для хранения информации о факультетах образовательной организации. Модель используется при распределении пользователей по факультетам и формировании статистических данных.

Описание методов класса Faculty представлено в таблице 4.2.

Таблица 4.2 – Методы класса Faculty (models/faculty.py)

Метод	Доступ	Описание
<code>__init__(self, name, code)</code>	public	Конструктор объекта факультета
<code>get_departments(self)</code>	public	Возвращает список кафедр факультета
<code>count_documents(self)</code>	public	Подсчитывает количество документов факультета

4.1.3 Класс Department (модель кафедры)

Класс Department (модуль models/department.py) предназначен для хранения информации о кафедрах образовательной организации. Модель используется при регистрации пользователей, распределении документов и формировании статистики.

Описание методов класса Department представлено в таблице 4.3.

Таблица 4.3 – Методы класса Department (models/department.py)

Метод	Доступ	Описание
<code>__init__(self, name, faculty_id)</code>	public	Конструктор объекта кафедры
<code>get_users(self)</code>	public	Возвращает список пользователей кафедры
<code>count_manuscripts(self)</code>	public	Подсчитывает количество документов кафедры

4.1.4 Класс Manual (модель методических указаний)

Класс Manual (модуль models/manual.py) предназначен для хранения информации о методических указаниях, загружаемых пользователями системы. Модель содержит сведения о файле, статусе проверки и результатах обработки документа.

Описание методов класса Manual представлено в таблице 4.4.

Таблица 4.4 – Методы класса Manual (models/manual.py)

Метод	Доступ	Описание
__init__(self, title, filename)	public	Конструктор объекта методических указаний
set_status(self, status)	public	Устанавливает статус обработки документа
get_report(self)	public	Возвращает путь к PDF-отчёту
mark_checked(self)	public	Помечает документ как проверенный

4.1.5 Класс Tutorial (модель учебного пособия)

Класс Tutorial (модуль models/tutorial.py) предназначен для хранения информации об учебных пособиях, загружаемых в систему. Модель используется модулем проверки документов и системой статистики.

Описание методов класса Tutorial представлено в таблице 4.5.

Таблица 4.5 – Методы класса Tutorial (models/tutorial.py)

Метод	Доступ	Описание
__init__(self, title, filename)	public	Конструктор объекта учебного пособия
set_status(self, status)	public	Устанавливает статус проверки документа

Продолжение таблицы 4.5

Метод	Доступ	Описание
generate_preview(self)	public	Формирует предварительную информацию о документе

4.1.6 Класс Monograph (модель монографии)

Класс Monograph (модуль models/monograph.py) предназначен для хранения сведений о монографиях, обрабатываемых системой. Модель используется при проверке документов и формировании статистических данных.

Описание методов класса Monograph представлено в таблице 4.6.

Таблица 4.6 – Методы класса Monograph (models/monograph.py)

Метод	Доступ	Описание
__init__(self, title, filename)	public	Конструктор объекта монографии
set_status(self, status)	public	Устанавливает статус обработки монографии
export_metadata(self)	public	Выполняет экспорт метаданных документа

4.1.7 Модуль проверки документов

Модуль проверки документов является основным компонентом информационно-аналитической системы и предназначен для автоматизированного анализа учебно-методических материалов на соответствие установленным требованиям оформления.

Модуль реализует загрузку DOCX-документов, их анализ, выявление ошибок форматирования, формирование визуальной разметки нарушений и генерацию итогового PDF-отчёта.

Основная логика модуля реализована в следующих компонентах:

1. checker.py — основной модуль проверки документов.

2. `highlight_errors.py` — модуль визуального выделения ошибок в документе.

3. `pdf_report.py` — модуль формирования PDF-отчётов.

4. `schemas/checker.py` — схемы валидации входных данных.

Описание функций модуля `checker.py` представлено в таблице 4.7.

Таблица 4.7 – Функции модуля `checker.py`

Функция	Доступ	Описание
<code>check_document(file_path)</code>	public	Выполняет комплексную проверку DOCX-документа
<code>check_margins(document)</code>	public	Проверяет параметры полей страницы
<code>check_fonts(document)</code>	public	Проверяет шрифты и размеры текста
<code>check_headings(document)</code>	public	Проверяет корректность заголовков
<code>check_paragraphs(document)</code>	public	Проверяет параметры абзацев и интервалов
<code>check_tables(document)</code>	public	Проверяет оформление таблиц
<code>check_images(document)</code>	public	Проверяет оформление изображений
<code>collect_errors(errors)</code>	private	Формирует итоговый список ошибок

Описание функций модуля `highlight_errors.py` представлено в таблице 4.8.

Таблица 4.8 – Функции модуля `highlight_errors.py`

Функция	Доступ	Описание
<code>highlight_document(input_path, output_path)</code>	public	Создаёт копию документа с выделением ошибок

Продолжение таблицы 4.8

Функция	Доступ	Описание
highlight_paragraph(paragraph)	private	Выделяет ошибочный абзац цветом
highlight_table(table)	private	Выделяет ошибки оформления таблиц
save_highlighted_document(path)	private	Сохраняет документ с разметкой ошибок

4.1.8 Модуль генерации титульных листов

Модуль генерации титульных листов предназначен для автоматизированного формирования титульной части учебно-методических документов в соответствии с установленными требованиями оформления.

Модуль реализует шаблонный подход к генерации DOCX-документов и обеспечивает автоматическое заполнение титульных страниц на основе пользовательских данных.

Основная логика генерации реализована в следующих компонентах:

1. title_page.py — маршруты API генерации титульных листов.
2. title_page_generator.py — модуль формирования DOCX-документа.
3. schemas/title_page.py — схемы проверки входных данных.

Описание функций модуля title_page.py представлено в таблице 4.9.

Таблица 4.9 – Функции модуля title_page.py

Функция	Доступ	Описание
generate_manual(data)	public	Генерирует титульный лист методических указаний
generate_tutorial(data)	public	Генерирует титульный лист учебного пособия
generate_monograph(data)	public	Генерирует титульный лист монографии

Продолжение таблицы 4.9

Функция	Доступ	Описание
validate_title_page(data)	private	Проверяет корректность входных данных

Описание функций модуля title_page_generator.py представлено в таблице 4.10.

Таблица 4.10 – Функции модуля title_page_generator.py

Функция	Доступ	Описание
create_document()	public	Создаёт структуру DOCX-документа
add_title_content(document, data)	public	Добавляет текстовые блоки титульного листа
apply_styles(document)	public	Применяет параметры форматирования
save_document(document, path)	public	Сохраняет итоговый DOCX-файл

4.1.9 Модуль формирования PDF-отчётов

Модуль формирования PDF-отчётов предназначен для автоматизированного создания структурированных отчётов по результатам проверки документов.

Модуль получает результаты анализа документа, выполняет группировку ошибок и формирует итоговый PDF-документ с информацией о найденных нарушениях.

Основная логика реализована в модуле pdf_report.py. Описание функций модуля pdf_report.py представлено в таблице 4.11.

Таблица 4.11 – Функции модуля pdf_report.py

Функция	Доступ	Описание
generate_pdf_report(errors, document_info)	public	Формирует PDF-отчёт по результатам проверки
group_errors(errors)	public	Выполняет группировку однотипных ошибок
build_statistics(errors)	public	Формирует статистику нарушений
create_report_header(pdf)	private	Создаёт заголовочную часть отчёта
create_error_table(pdf, errors)	private	Формирует таблицу ошибок
save_pdf(path)	private	Сохраняет итоговый PDF-документ

4.2 Системное тестирование

Системное тестирование информационно-аналитической системы проводилось в соответствии со сценариями использования, определёнными в техническом задании. В процессе тестирования проверялись основные функции системы: генерация титульных листов, автоматическая проверка документов и формирование PDF-отчётов.

Тестирование выполнялось в операционной системе Windows 11 с использованием веб-браузера Google Chrome. Корректность работы системы подтверждена результатами выполнения тест-кейсов и соответствующими скриншотами интерфейса.

4.2.1 Сводная таблица тест-кейсов

Сводная информация о выполненных тест-кейсах представлена в таблице 4.12.

Таблица 4.12 – Сводная таблица тест-кейсов

ID	Название тест-кейса	Результат
ТС-01	Генерация титульного листа учебного издания	Пройден
ТС-02	Автоматическая проверка DOCX-документа	Пройден

Продолжение таблицы 4.12

ID	Название тест-кейса	Результат
ТС-03	Формирование PDF-отчёта о результатах проверки	Пройден
ТС-04	Скачивание сформированного PDF-отчёта	Пройден

4.2.2 ТС-01. Генерация титульного листа учебного издания

Предусловие: пользователь авторизован в системе и открыта страница генерации титульных листов.

Шаги:

1. Выбрать тип документа.
2. Заполнить поля формы генерации.
3. Нажать кнопку «Сгенерировать».

Ожидаемый результат: система формирует DOCX-документ с титульным листом и автоматически предоставляет файл для скачивания.

Фактический результат: пройден.

На рисунке 4.1 представлен интерфейс генерации титульного листа.

Рисунок 4.1 – Интерфейс генерации титульного листа

На рисунке 4.2 представлен результат генерации титульного листа.

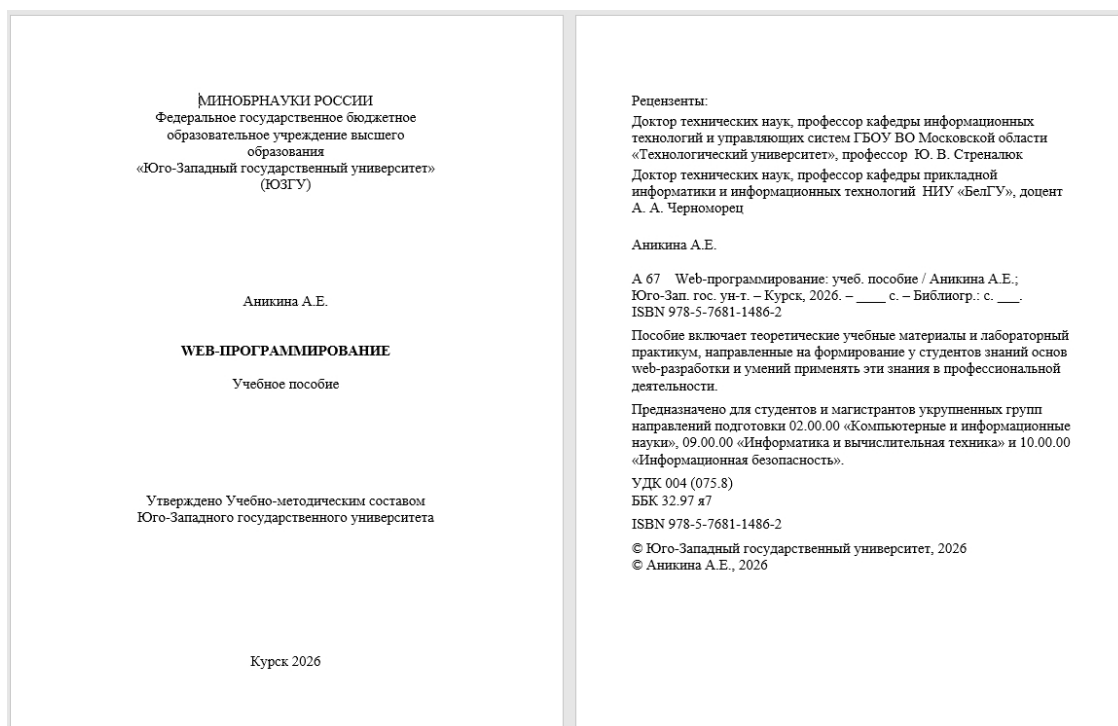


Рисунок 4.2 – Результат генерации титульного листа

4.2.3 ТС-02. Автоматическая проверка DOCX-документа

Предусловие: пользователь авторизован и открыта страница проверки документов.

Шаги:

1. Загрузить DOCX-документ.
2. Запустить проверку документа.

Ожидаемый результат: система выполняет анализ документа, определяет ошибки оформления и отображает результаты проверки.

Фактический результат: пройден.

На рисунке 4.3 представлен пример разметки при проверке документа.

1. **Backend** (бэкенд или серверная часть приложения). Работает на удаленном компьютере, который может находиться, где угодно. Она может быть написана на разных языках программирования: PHP, Python, Ruby, C# и других. Если создавать приложение, используя только серверную часть, то в результате любых переходов между разделами, отправок форм, обновления данных, сервером будет генерироваться новый HTML-файл и страница в браузере будет перезагружаться.

2. **Frontend** (фронтенд или клиентская часть приложения). Выполняется в браузере пользователя. Эта часть написана на языке программирования JavaScript. Приложение может состоять только из клиентской части, если не требуется хранить данные пользователя дольше одной сессии. Это могут быть, например, фоторедакторы или простые игрушки.

3. **Single page application** (SPA или одностраничное приложение). Более интересный вариант, когда используются и бэкенд и фронтенд. С помощью их взаимодействия можно создать приложение, которое будет работать совсем без перезагрузок страницы в браузере. Или в упрощенном варианте, когда переходы между разделами вызывают перезагрузки, но любые действия в разделе обходятся без них.

Большинство web-приложений имеют архитектуру *клиентсервер*. При этом в качестве клиентской программы используется web-браузер, а обмен с серверной частью происходит с использованием протокола HTTP.

Как показано на рисунке 1, web-сервер (блок, реализующий

Примечания

16 мин. назад

В основном тексте используется неверный размер шрифта. Необходимо установить размер шрифта 16 pt для всего основного текста документа.

Обнаружен неверный абзацный отступ первой строки. Необходимо установить отступ первой строки 1,25 см.

Обнаружен неверный межстрочный интервал. Необходимо установить одинарный межстрочный интервал для основного текста документа.

Рисунок 4.3 – Пример разметки при проверке документа

4.2.4 ТС-03. Формирование PDF-отчёта о результатах проверки

Предусловие: проверка документа успешно завершена.

Шаги:

1. Выполнить проверку документа.
2. Нажать кнопку формирования отчёта.

Ожидаемый результат: система автоматически формирует PDF-отчёт с перечнем найденных ошибок.

Фактический результат: пройден.

На рисунке 4.4 представлен пример сформированного PDF-отчёта.

Отчёт о проверке

Файл: Anikina_Web-programmirovanie_Uchebnoe_posobie1.docx

Дата проверки: 06.05.2026 16:57:23

Ошибки: 61 | Предупреждения: 0

Статус: документ содержит критические ошибки

Найденные замечания

#	Тип	Где	Описание
1	Ошибка	Стр. 2 - Стр. 2	В основном тексте используется неверный шрифт. Необходимо установить шрифт Times New Roman для всего основного текста документа.
2	Ошибка	Стр. 6 - Стр. 6	В основном тексте используется неверный шрифт. Необходимо установить шрифт Times New Roman для всего основного текста документа.
3	Ошибка	Стр. 33 - Стр. 33	В основном тексте используется неверный шрифт. Необходимо установить шрифт Times New Roman для всего основного текста документа.
4	Ошибка	Стр. 35 - Стр. 44	В основном тексте используется неверный шрифт. Необходимо установить шрифт Times New Roman для всего основного текста документа.
5	Ошибка	Стр. 46 - Стр. 69	В основном тексте используется неверный шрифт. Необходимо установить шрифт Times New Roman для всего основного текста документа.

Рисунок 4.4 – PDF-отчёт о результатах проверки

4.2.5 ТС-04. Скачивание сформированного PDF-отчёта

Предусловие: PDF-отчёт успешно сформирован.

Шаги:

1. Нажать кнопку скачивания отчёта.

Ожидаемый результат: пользователю загружается PDF-файл с результатами проверки документа.

Фактический результат: пройден.

4.2.6 Результаты тестирования

В ходе тестирования были успешно проверены основные функции информационно-аналитической системы. Все тест-кейсы завершились успешно, ошибки в работе системы выявлены не были.

Результаты тестирования подтвердили:

- корректность генерации титульных листов;
- правильность проверки DOCX-документов;

- корректность формирования PDF-отчётов;
- стабильность взаимодействия клиентской и серверной части системы.

Система соответствует требованиям технического задания и готова к эксплуатации.

4.2.7 Нефункциональное тестирование

Помимо функционального тестирования были проверены нефункциональные требования, предъявляемые к информационно-аналитической системе.

Быстродействие. Проверка DOCX-документов объёмом до 100 страниц выполняется в среднем не более чем за 15–20 секунд. Генерация титульных листов и формирование PDF-отчётов занимают менее 5 секунд. Загрузка страниц веб-интерфейса выполняется без заметных задержек.

Корректность логирования. Система фиксирует основные события работы приложения:

- авторизацию пользователей;
- ошибки проверки документов;
- генерацию титульных листов;
- формирование PDF-отчётов;
- ошибки серверной части.

Записи сохраняются в журнале с указанием даты и времени возникновения события.

Совместимость с браузерами. Клиентская часть системы протестирована в браузерах Google Chrome, Mozilla Firefox и Microsoft Edge. Во всех браузерах корректно отображаются элементы интерфейса, формы загрузки файлов и результаты проверки документов.

Надёжность. При возникновении ошибок обработки документов система корректно завершает выполнение операций и предотвращает повреждение данных. При повторном запуске приложения работоспособность системы полностью восстанавливается.

Безопасность. Доступ к основным функциям системы предоставляется только авторизованным пользователям. Реализована проверка JWT-токенов и разграничение прав доступа между обычными пользователями и администраторами.

Все нефункциональные требования выполнены в полном объёме.

4.2.8 Вывод по результатам тестирования

В ходе системного тестирования были успешно проверены функциональные и нефункциональные возможности информационно-аналитической системы.

Результаты тестирования подтвердили:

- корректность проверки учебно-методических документов;
- правильность генерации титульных листов;
- корректность формирования PDF-отчётов;
- стабильность взаимодействия клиентской и серверной части;
- соответствие системы требованиям технического задания.

Разработанная информационно-аналитическая система функционирует стабильно, обеспечивает автоматизацию основных процессов редакционно-издательского отдела и готова к практической эксплуатации.

ЗАКЛЮЧЕНИЕ

В результате выполнения выпускной квалификационной работы была разработана информационно-аналитическая система автоматизации проверки учебно-методических материалов редакционно-издательского отдела. Разработанная система позволяет выполнять автоматизированную проверку документов на соответствие требованиям оформления, формировать отчёты о выявленных нарушениях, генерировать титульные листы и обеспечивать централизованное хранение данных.

В ходе работы был проведён анализ предметной области, связанной с организацией процессов проверки и сопровождения учебно-методической документации. Особое внимание было уделено проблемам ручной проверки документов, требованиям к оформлению учебных материалов и существующим подходам к автоматизации документооборота. Выполненный анализ позволил сформировать требования к функциональности разрабатываемой системы.

Разработанная система реализует следующие основные возможности:

- аутентификацию и авторизацию пользователей;
- загрузку и обработку учебно-методических документов;
- автоматическую проверку параметров форматирования;
- формирование PDF-отчётов по результатам проверки;
- генерацию титульных листов документов;
- взаимодействие пользователей с редакционно-издательским отделом через веб-интерфейс.

Для реализации системы использована клиент-серверная архитектура. Серверная часть разработана с использованием FastAPI и PostgreSQL, а клиентская часть реализована с применением HTML, CSS и JavaScript. Для хранения и обработки данных использована технология ORM SQLAlchemy.

В процессе разработки было проведено тестирование системы, включающее проверку корректности обработки документов, формирования отчётов, работы механизмов аутентификации и взаимодействия клиентской и сервер-

ной части. Результаты тестирования подтвердили корректность функционирования разработанной системы и соответствие реализованного функционала поставленным требованиям.

Основные результаты выпускной квалификационной работы:

1. Проведён анализ процессов проверки учебно-методических материалов и существующих средств автоматизации документооборота.
2. Разработана архитектура информационно-аналитической системы.
3. Реализована серверная часть системы с использованием FastAPI и PostgreSQL.
4. Разработана клиентская часть веб-приложения.
5. Реализованы модули проверки документов, генерации титульных листов и формирования PDF-отчётов.
6. Проведено тестирование системы и подтверждена корректность её работы.

Все задачи, поставленные в начале выпускной квалификационной работы, были успешно решены. Таким образом, цель выпускной квалификационной работы — разработка информационно-аналитической системы автоматизации проверки учебно-методических материалов редакционно-издательского отдела — была достигнута.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Лутц, М. Изучаем Python. Том 1 / М. Лутц. – 5-е изд. – Санкт-Петербург : Питер, 2019. – 832 с. – ISBN 978-5-907144-52-1. – Текст : непосредственный.
2. Рамальо, Л. Python. К вершинам мастерства / Л. Рамальо. – Москва : ДМК Пресс, 2017. – 768 с. – ISBN 978-5-97060-384-3. – Текст : непосредственный.
3. Python Documentation : официальный сайт. – URL: <https://docs.python.org/3/> (дата обращения: 10.01.2026).
4. PEP 8 – Style Guide for Python Code : официальный сайт. – URL: <https://peps.python.org/pep-0008/> (дата обращения: 15.01.2026).
5. Threading — Thread-based parallelism // Python Documentation : официальный сайт. – URL: <https://docs.python.org/3/library/threading.html> (дата обращения: 20.01.2026).
6. Logging HOWTO // Python Documentation : официальный сайт. – URL: <https://docs.python.org/3/howto/logging.html> (дата обращения: 25.01.2026).
7. Буэно, С. Разработка API с FastAPI / С. Буэно. – Москва : ДМК Пресс, 2022. – 384 с. – ISBN 978-5-97060-948-8. – Текст : непосредственный.
8. FastAPI Documentation : официальный сайт. – URL: <https://fastapi.tiangolo.com/> (дата обращения: 01.02.2026).
9. Pydantic Documentation : официальный сайт. – URL: <https://docs.pydantic.dev/> (дата обращения: 05.02.2026).
10. Uvicorn Documentation : официальный сайт. – URL: <https://www.uvicorn.org/> (дата обращения: 10.02.2026).
11. Starlette Documentation : официальный сайт. – URL: <https://www.starlette.io/> (дата обращения: 14.02.2026).
12. Дейт, К. Дж. Введение в системы баз данных / К. Дж. Дейт. – 8-е изд. – Москва : Вильямс, 2005. – 1328 с. – ISBN 978-5-8459-0788-2. – Текст : непосредственный.

13. Коннолли, Т. Базы данных. Проектирование, реализация и сопровождение. Теория и практика / Т. Коннолли, К. Бегг. – 3-е изд. – Москва : Вильямс, 2003. – 1440 с. – ISBN 978-5-8459-0527-7. – Текст : непосредственный.
14. Грофф, Дж. Р. SQL: полное руководство / Дж. Р. Грофф, П. Н. Вайнберг, Э. Дж. Оппель. – 3-е изд. – Санкт-Петербург : Диалектика, 2020. – 960 с. – ISBN 978-5-907144-26-5. – Текст : непосредственный.
15. Моргунов, Е. П. PostgreSQL. Основы языка SQL / Е. П. Моргунов. – Санкт-Петербург : БХВ-Петербург, 2022. – 336 с. – ISBN 978-5-9775-3960-9. – Текст : непосредственный.
16. PostgreSQL Documentation : официальный сайт. – URL: <https://www.postgresql.org/docs/> (дата обращения: 18.02.2026).
17. SQLAlchemy Documentation : официальный сайт. – URL: <https://docs.sqlalchemy.org/> (дата обращения: 22.02.2026).
18. Alembic – Database Migrations for SQLAlchemy : официальный сайт. – URL: <https://alembic.sqlalchemy.org/> (дата обращения: 26.02.2026).
19. Psycopg – PostgreSQL adapter for Python : официальный сайт. – URL: <https://www.psycopg.org/docs/> (дата обращения: 01.03.2026).
20. python-docx Documentation : официальный сайт. – URL: <https://python-docx.readthedocs.io/> (дата обращения: 05.03.2026).
21. pywin32 Documentation // GitHub : официальный сайт. – URL: <https://github.com/mhammond/pywin32> (дата обращения: 09.03.2026).
22. ECMA-376 Office Open XML File Formats : официальный сайт. – URL: <https://www.ecma-international.org/publications-and-standards/standards/ecma-376/> (дата обращения: 13.03.2026).
23. JWT Introduction : официальный сайт. – URL: <https://jwt.io/introduction> (дата обращения: 17.03.2026).
24. The OAuth 2.0 Authorization Framework : официальный сайт. – URL: <https://datatracker.ietf.org/doc/html/rfc6749> (дата обращения: 21.03.2026).
25. Мартин, Р. Чистый код: создание, анализ и рефакторинг / Р. Мартин. – Санкт-Петербург : Питер, 2013. – 464 с. – ISBN 978-5-496-00487-9. – Текст : непосредственный.

26. Мартин, Р. Чистая архитектура. Искусство разработки программного обеспечения / Р. Мартин. – Санкт-Петербург : Питер, 2018. – 352 с. – ISBN 978-5-4461-0772-8. – Текст : непосредственный.
27. Рамбо, Дж. UML 2.0. Объектно-ориентированное моделирование и разработка / Дж. Рамбо, М. Блаха. – 2-е изд. – Санкт-Петербург : Питер, 2021. – 542 с. – ISBN 978-5-4461-9428-5. – Текст : непосредственный.
28. Unified Modeling Language Specification, Version 2.5.1 : официальный сайт. – URL: <https://www.omg.org/spec/UML/2.5.1/PDF> (дата обращения: 25.03.2026).
29. draw.io Documentation : официальный сайт. – URL: <https://www.drawio.com/doc/> (дата обращения: 28.03.2026).
30. Чакон, С. Git для профессионального программиста / С. Чакон, Б. Штрауб. – Санкт-Петербург : Питер, 2016. – 496 с. – ISBN 978-5-496-01763-3. – Текст : непосредственный.
31. Git Documentation : официальный сайт. – URL: <https://git-scm.com/doc> (дата обращения: 31.03.2026).
32. Дзекетт, Д. HTML и CSS. Разработка и создание веб-сайтов / Д. Дзекетт. – Москва : Эксмо, 2014. – 480 с. – ISBN 978-5-699-64193-2. – Текст : непосредственный.
33. Дзекетт, Д. JavaScript и jQuery. Интерактивная web-разработка / Д. Дзекетт. – Москва : Эксмо, 2017. – 640 с. – ISBN 978-5-699-80285-2. – Текст : непосредственный.
34. Фрейн, Б. HTML5 и CSS3. Разработка современных web-сайтов / Б. Фрейн. – 2-е изд. – Санкт-Петербург : Питер, 2022. – 384 с. – ISBN 978-5-4461-0526-2. – Текст : непосредственный.
35. HTML // MDN Web Docs : официальный сайт. – URL: <https://developer.mozilla.org/ru/docs/Web/HTML> (дата обращения: 02.04.2026).
36. CSS // MDN Web Docs : официальный сайт. – URL: <https://developer.mozilla.org/ru/docs/Web/CSS> (дата обращения: 04.04.2026).
37. JavaScript // MDN Web Docs : официальный сайт. – URL: <https://developer.mozilla.org/ru/docs/Web/JavaScript> (дата обращения: 06.04.2026).

38. HTTP // MDN Web Docs : официальный сайт. – URL: <https://developer.mozilla.org/ru/docs/Web/HTTP> (дата обращения: 08.04.2026).
39. Fielding, R. HTTP Semantics / R. Fielding, M. Nottingham, J. Reschke. – RFC 9110. – URL: <https://www.rfc-editor.org/rfc/rfc9110.html> (дата обращения: 10.04.2026).
40. Fielding, R. Architectural Styles and the Design of Network-based Software Architectures // University of California : официальный сайт. – URL: https://www.ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm (дата обращения: 12.04.2026).
41. Оккен, Б. pytest. Быстрое тестирование приложений на Python / Б. Оккен. – Санкт-Петербург : Питер, 2020. – 320 с. – ISBN 978-5-4461-1265-4. – Текст : непосредственный.
42. PyCharm Documentation : официальный сайт. – URL: <https://www.jetbrains.com/pycharm/documentation/> (дата обращения: 14.04.2026).
43. Bootstrap Documentation : официальный сайт. – URL: <https://getbootstrap.com/docs/5.3/> (дата обращения: 16.04.2026).
44. Docker Documentation : официальный сайт. – URL: <https://docs.docker.com/> (дата обращения: 18.04.2026).
45. NGINX Documentation : официальный сайт. – URL: <https://nginx.org/en/docs/> (дата обращения: 20.04.2026).
46. ReportLab Documentation : официальный сайт. – URL: <https://docs.reportlab.com/> (дата обращения: 22.04.2026).
47. asyncio Documentation // Python Documentation : официальный сайт. – URL: <https://docs.python.org/3/library/asyncio.html> (дата обращения: 24.04.2026).
48. Кнут, Д. Э. Искусство программирования. Том 1. Основные алгоритмы / Д. Э. Кнут. – 4-е изд. – Москва : Вильямс, 2022. – 720 с. – ISBN 978-5-8459-2075-1. – Текст : непосредственный.
49. Лопес, Ф. SQLAlchemy. Архитектура и паттерны / Ф. Лопес. – Москва : ДМК Пресс, 2020. – 384 с. – ISBN 978-5-97060-765-0. – Текст : непосредственный.

50. Любанович, Б. Простой Python. Современный стиль программирования / Б. Любанович. – Санкт-Петербург : Питер, 2020. – 480 с. – ISBN 978-5-4461-1269-2. – Текст : непосредственный.
51. Вурс, Д. Python. Карманный справочник / Д. Вурс. – Москва : Эксмо, 2022. – 320 с. – ISBN 978-5-04-164234-5. – Текст : непосредственный.
52. Python Tutorial // Python Documentation : официальный сайт. – URL: <https://docs.python.org/3/tutorial/> (дата обращения: 26.04.2026).
53. Рамальо, Л. Python. К вершинам мастерства. Том 2 / Л. Рамальо. – Москва : ДМК Пресс, 2023. – 672 с. – ISBN 978-5-97060-985-3. – Текст : непосредственный.
54. Дауни, А. Б. Think Python. Аналитический подход к программированию / А. Б. Дауни. – 3-е изд. – Санкт-Петербург : Питер, 2024. – 320 с. – ISBN 978-5-4461-2154-4. – Текст : непосредственный.

ПРИЛОЖЕНИЕ А

Представление графического материала

Графический материал, выполненный на отдельных листах, изображен на рисунках А.1–А.11.

Сведения о ВКРБ

Минобрнауки России

Юго-Западный государственный университет

Кафедра программной инженерии

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА ПО ПРОГРАММЕ БАКАЛАВРИАТА

Бизнес-проект “Информационно-аналитическая система для РИО вуза.
Разработка сервиса

Руководитель ВКРБ
к.т.н, доцент
Аникина Елена Игоревна

Автор ВКРБ
студент группы ПО-22б
Киреев Станислав Андреевич

ВКРБ 2206509.09.03.04.26.010			Дат.	Испол.	Исполн.
Сведения о ВКРБ			Лист 1	Лист 2	Лист 3
Выпускная квалификационная работа бакалавра			03/У ПО-22б		
Аннотация	Формат А. 0	Получено			
Аннотация	Формат А. 0	Получено			
Аннотация	Формат А. 0	Получено			

Рисунок А.1 – Сведения о ВКРБ

Цель и задачи разработки

Цель настоящей работы - разработка информационно-аналитической системы автоматизации учёта и проверки учебно-методических материалов кафедры, обеспечивающей централизованную обработку документов, автоматическую проверку их оформления и формирование сопроводительной документации.

Для достижения поставленной цели необходимо решить следующие задачи:

- спроектировать архитектуру серверной части информационно-аналитической системы с использованием Python, FastAPI и PostgreSQL, обеспечив модульную структуру приложения и обработку пользовательских запросов;
- реализовать механизм регистрации, авторизации и разграничения прав доступа пользователей;
- разработать интерфейсы (API) для загрузки, хранения, обработки и управления учебно-методическими документами различных типов;
- организовать автоматизированную проверку DOCX-документов на соответствие требованиям оформления;
- реализовать механизм визуального выделения ошибок форматирования в документе с формированием копии DOCX-файла, содержащей пометки нарушений;
- разработать модуль генерации титульных листов для различных видов учебно-методических материалов;
- обеспечить автоматическое формирование PDF-отчётов по результатам проверки документов;
- внедрить механизм хранения информации о пользователях, кафедрах, факультетах и документах с использованием PostgreSQL;
- спроектировать обработку исключений и ошибок системы с возвратом корректных HTTP-ответов, логированием событий и обеспечением отказоустойчивости серверной части;
- провести модульное и системное тестирование основных компонентов информационно-аналитической системы, включая проверку корректности обработки документов, генерации отчётов, работы механизмов авторизации и взаимодействия с базой данных.

			ВКР5 2206509.09.03.04.26.010		
	Фамилия И. О.	Подпись/Дата	Цель и задачи разработки	Лист	Листов
Аспирант	Александр С.А.			Лист 2	Листов 11
Руководитель	Александр С.А.		Выпускная квалификационная работа бакалавра		
Проверитель	Степанов А.А.				

Рисунок А.2 – Цели и задачи разработки



Рисунок А.3 – Диаграмма прецедентов пользователя



Рисунок А.4 – Общая схема архитектуры



Рисунок А.5 – Архитектура серверной части

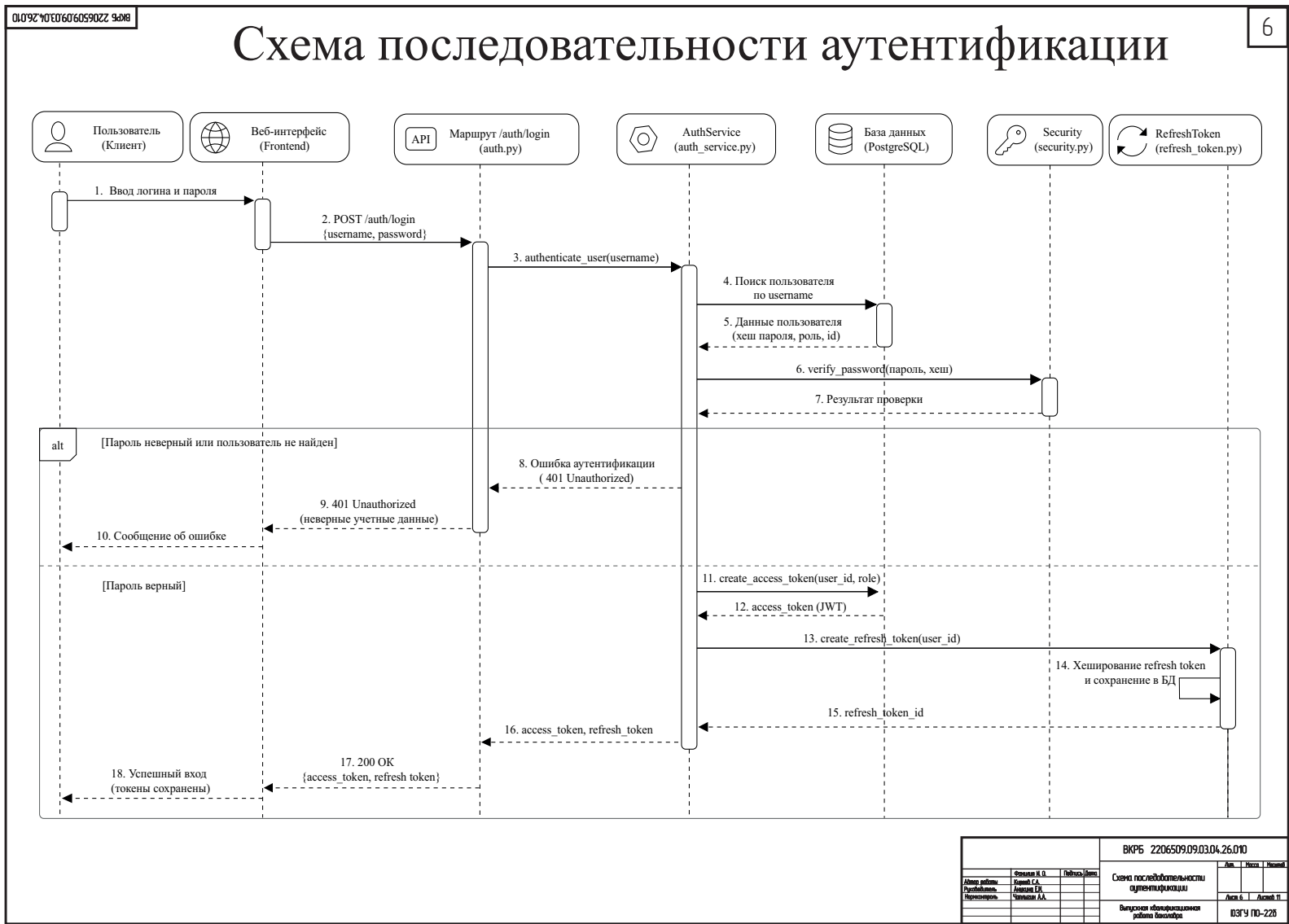


Рисунок А.6 – Схема последовательности аутентификации



Рисунок А.7 – Структура маршрутизации серверной части

01.07.2022 22:06:50 9888

Страница авторизации в приложении

8

Проверка методичек
Веб-система

Авторизация

Профиль

Проверка

Титульный лист

Статистика

Администрирование

Статус
Авторизован

Система автономной проверки файлов
Загрузка файлов, проверка оформления, PDF-отчет и статистика

[Обновить профиль](#)
Выйти

Авторизация

Вход

Email

Пароль

Войти

Регистрация

ФИО

Email

Пароль

Код факультета

Название кафедры

Роль

Зарегистрироваться

ВКРБ 2206509.09.03.04.26.010			
Авторизация Проверка Статистика	Страница авторизации в приложении	Лист 8	Лист 11
Высшая квалификационная комиссия		03/У ПО-228	

Рисунок А.8 – Страница авторизации в приложении

010792740303090903.04.26.010

9

Страница проверки файлов

Ю

Проверка методичек
Веб-система

Авторизация

Профиль

Проверка

Титульный лист

Статистика

Администрирование

Статус
Авторизован

Система автономной проверки файлов
Загрузка файлов, проверка оформления, PDF-отчёт и статистика

[Обновить профиль](#)
[Выйти](#)

Система проверки файлов

Выберите .doc или .docx файл

Выберите файл
Файл не выбран

[Проверить файл](#)

Здесь появится результат проверки

Мои файлы

[Обновить список](#)

Не удалось загрузить список методичек

				ВКРБ 2206509.09.03.04.26.010			
	Фамилия И. О.	Полное имя		Страница проверки файлов		Лит	Носит
Адрес работы	Фамилия С.А.	Имя С.А.	Отчество С.А.			Лист 9	Листов 11
Должность	Имя С.А.	Имя С.А.	Имя С.А.				
Период работы	Имя С.А.	Имя С.А.	Имя С.А.				
				Выпускная квалификационная работа бакалавра		037-У ПО-226	

Рисунок А.9 – Страница проверки файлов

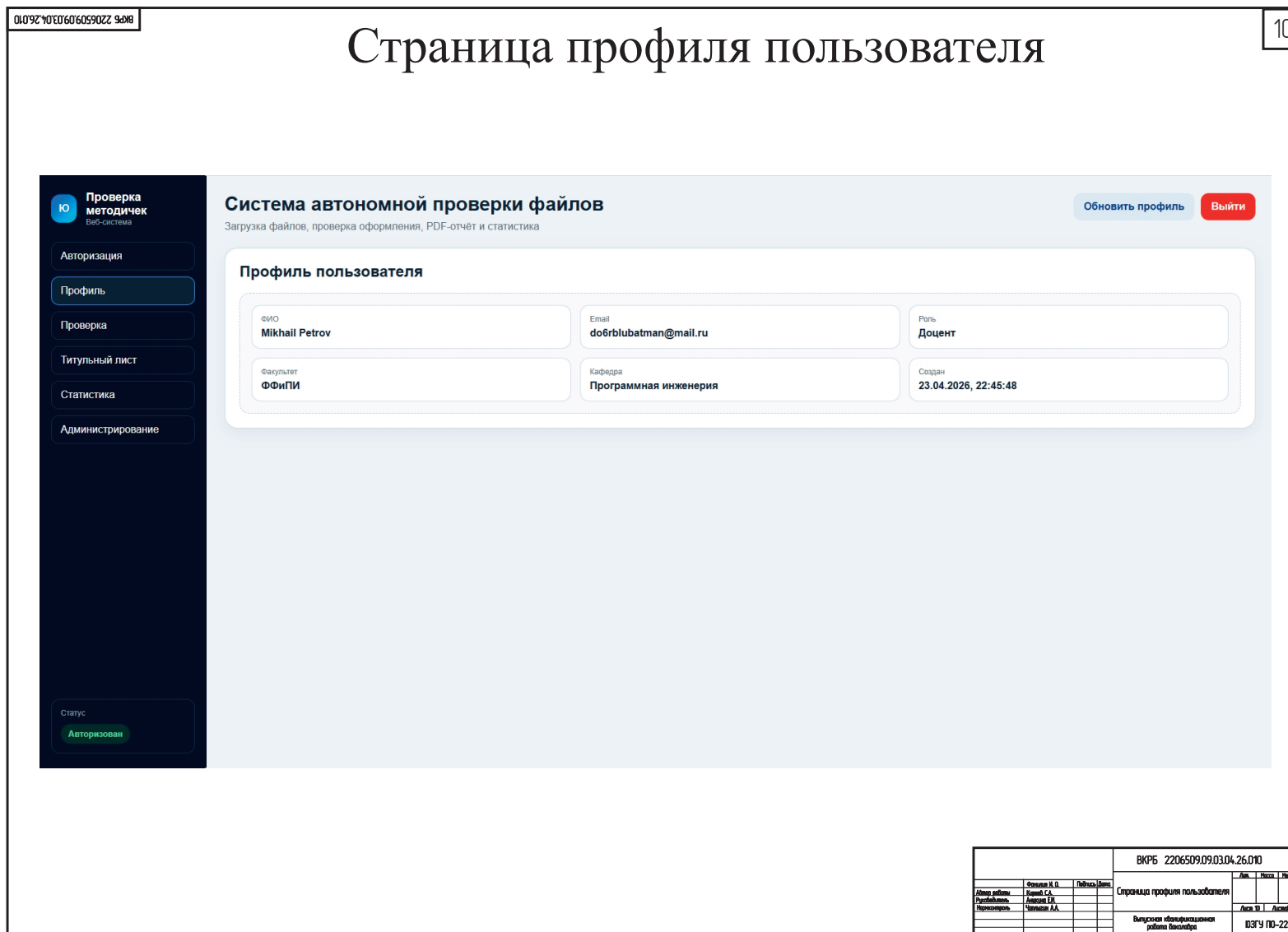


Рисунок А.10 – Страница профиля пользователя

Разработанная система облегчает работу сотрудников кафедры и сотрудников РИО. Пользователи имеют возможность загружать документы, выполнять автоматическую проверку оформления, формировать титульные листы и получать PDF-отчёты с результатами анализа. Администратор системы может управлять пользователями, факультетами, кафедрами и контролировать обработку документов.

1. Проведён анализ предметной области и определены требования к информационно-аналитической системе.
2. Разработана архитектура серверной части системы с использованием Python, FastAPI и PostgreSQL.
3. Реализованы механизмы регистрации, авторизации и разграничения прав доступа пользователей на основе JWT-токенов.
4. Разработан модуль автоматической проверки DOCX-документов на соответствие требованиям оформления, включая проверку шрифтов, полей страницы, заголовков, таблиц и изображений.
5. Реализован механизм визуального выделения ошибок форматирования и формирования PDF-отчётов по результатам проверки документов.
6. Разработан модуль генерации титульных листов для различных типов учебно-методических изданий.
7. Проведено модульное и системное тестирование основных компонентов системы.

		ВКРБ 2206509.09.03.04.26.010	
	Фамилия И.О.	Подпись	Дата
Адрес работы	Ковалев С.А.		
Российская	Александр Е.И.		
Переселенцев	Чайкина А.А.		
		Заключение	
		Выдана идентификация	03.07.2012
		работы	
		исполнения	

Рисунок А.11 – Заключение

ПРИЛОЖЕНИЕ Б

Фрагменты исходного кода программы

auth.py

```
1 from fastapi import APIRouter, Depends, Request
2 from sqlalchemy.orm import Session
3
4 from app.db.session import get_db
5 from app.models.department import Department
6 from app.models.faculty import Faculty
7 from app.models.user import User
8 from app.schemas.auth import UserLogin, UserRegister
9 from app.schemas.token import MessageResponse, RefreshTokenRequest, TokenPair
10 from app.schemas.user import UserInfo
11 from app.service.auth_service import (
12     login_user,
13     logout_all_user_sessions,
14     logout_user,
15     refresh_user_token,
16     register_user,
17 )
18 from app.utils.dependencies import get_current_user
19
20 router = APIRouter(prefix="/auth", tags=["auth"])
21
22
23 @router.post("/register", response_model=UserInfo)
24 def register(user_data: UserRegister, db: Session = Depends(get_db)):
25     user = register_user(
26         db=db,
27         fio=user_data.fio,
28         email=user_data.email,
29         password=user_data.password,
30         faculty_code=user_data.faculty_code,
31         department_code=user_data.department_code,
32         role=user_data.role,
33     )
34
35     faculty = db.query(Faculty).filter(Faculty.id_faculty == user.id_faculty)
36     .first()
37     department = db.query(Department).filter(Department.id_department == user
38     .id_department).first()
39
40     return UserInfo(
41         id_user=user.id_user,
42         fio=user.fio,
43         email=user.email,
44         role=user.role,
45         faculty_name=faculty.faculty_name if faculty else "",
46         department_name=department.department_name if department else "",
47         created_at=user.created_at,
```

```

48
49 @router.post("/login", response_model=TokenPair)
50 def login(user_data: UserLogin, request: Request, db: Session = Depends(
    get_db)):
51     access_token, refresh_token = login_user(
52         db=db,
53         email=user_data.email,
54         password=user_data.password,
55         request=request,
56     )
57
58     return TokenPair(
59         access_token=access_token,
60         refresh_token=refresh_token,
61         token_type="bearer",
62     )
63
64
65 @router.post("/refresh", response_model=TokenPair)
66 def refresh_tokens(data: RefreshTokenRequest, db: Session = Depends(get_db)):
67     new_access_token = refresh_user_token(db=db, refresh_token=data.
        refresh_token)
68
69     return TokenPair(
70         access_token=new_access_token,
71         refresh_token=data.refresh_token,
72         token_type="bearer",
73     )
74
75
76 @router.post("/logout", response_model=MessageResponse)
77 def logout(data: RefreshTokenRequest, db: Session = Depends(get_db)):
78     logout_user(db=db, refresh_token=data.refresh_token)
79     return MessageResponse(message="Выход выполнен успешно")
80
81
82 @router.post("/logout-all", response_model=MessageResponse)
83 def logout_all(
84     current_user: User = Depends(get_current_user),
85     db: Session = Depends(get_db),
86 ):
87     logout_all_user_sessions(db=db, user_id=current_user.id_user)
88     return MessageResponse(message="Выход со всех устройств выполнен успешно")
89
90
91 @router.get("/me", response_model=UserInfo)
92 def me(current_user: User = Depends(get_current_user), db: Session = Depends(
    get_db)):
93     faculty = db.query(Faculty).filter(Faculty.id_faculty == current_user.
        id_faculty).first()
94     department = db.query(Department).filter(Department.id_department ==
        current_user.id_department).first()
95

```

```

96     return UserInfo(
97         id_user=current_user.id_user,
98         fio=current_user.fio,
99         email=current_user.email,
100        role=current_user.role,
101        faculty_name=faculty.faculty_name if faculty else "",
102        department_name=department.department_name if department else "",
103        created_at=current_user.created_at,
104    )

```

checker.py

```

1  import hashlib
2  import os
3  import shutil
4  import tempfile
5  from uuid import uuid4
6
7  from fastapi import APIRouter, Depends, File, HTTPException, Request,
    UploadFile
8  from fastapi.responses import FileResponse
9  from sqlalchemy.orm import Session
10
11 from app.db.session import get_db
12 from app.models.manual import Manual
13 from app.models.user import User
14 from app.service.checker import WordMethodicalChecker
15 from app.service.pdf_report import generate_error_report_pdf
16 from app.utils.dependencies import get_current_user
17
18 router = APIRouter(prefix="/checker", tags=["checker"])
19
20 BASE_DIR = os.getcwd()
21 CHECKED_DIR = os.path.join(BASE_DIR, "checked_files")
22 REPORTS_DIR = os.path.join(BASE_DIR, "reports")
23
24 os.makedirs(CHECKED_DIR, exist_ok=True)
25 os.makedirs(REPORTS_DIR, exist_ok=True)
26
27
28 def calculate_file_hash(file_path: str) -> str:
29     sha256 = hashlib.sha256()
30     with open(file_path, "rb") as f:
31         for chunk in iter(lambda: f.read(8192), b""):
32             sha256.update(chunk)
33     return sha256.hexdigest()
34
35
36 @router.get("/ping")
37 def ping():
38     return {"message": "checker router works"}
39
40
41 @router.get("/download-checked/{filename}")
42 def download_checked_file(filename: str):

```

```

43 file_path = os.path.join(CHECKED_DIR, filename)
44
45 if not os.path.exists(file_path):
46     raise HTTPException(status_code=404, detail="Файл не найден")
47
48 ext = os.path.splitext(file_path)[1].lower()
49 media_type = (
50     "application/msword"
51     if ext == ".doc"
52     else "application/vnd.openxmlformats-officedocument.wordprocessingml.
        document"
53 )
54
55 return FileResponse(
56     path=file_path,
57     filename=filename,
58     media_type=media_type,
59 )
60
61
62 @router.get("/report/{filename}")
63 def open_pdf_report(filename: str):
64     file_path = os.path.join(REPORTS_DIR, filename)
65
66     if not os.path.exists(file_path):
67         raise HTTPException(status_code=404, detail="PDF-отчёт не найден")
68
69     return FileResponse(
70         path=file_path,
71         filename=filename,
72         media_type="application/pdf",
73     )
74
75
76 @router.post("/check")
77 async def check_document(
78     request: Request,
79     file: UploadFile = File(...),
80     current_user: User = Depends(get_current_user),
81     db: Session = Depends(get_db),
82 ):
83     if not file.filename:
84         raise HTTPException(status_code=400, detail="Файл не выбран")
85
86     if not file.filename.lower().endswith((".doc", ".docx")):
87         raise HTTPException(status_code=400, detail="Поддерживаются только .
            doc и .docx")
88
89     temp_dir = tempfile.mkdtemp()
90     temp_path = os.path.join(temp_dir, file.filename)
91
92     try:
93         with open(temp_path, "wb") as buffer:
94             shutil.copyfileobj(file.file, buffer)

```

```

95
96     with WordMethodicalChecker(visible=False, mark_document=True) as
97         checker:
98             report, checked_path = checker.check(temp_path)
99
100     if not checked_path or not os.path.exists(checked_path):
101         raise HTTPException(status_code=500, detail="Не удалось создать
102             проверенный файл")
103
104     unique_id = uuid4().hex
105     checked_filename = f"{unique_id}_{os.path.basename(checked_path)}"
106     saved_checked_path = os.path.join(CHECKED_DIR, checked_filename)
107     shutil.copy2(checked_path, saved_checked_path)
108
109     has_problems = len(report.issues) > 0
110
111     pdf_report_url = None
112
113     if has_problems:
114         pdf_filename = f"{unique_id}_report.pdf"
115         pdf_path = os.path.join(REPORTS_DIR, pdf_filename)
116
117         generate_error_report_pdf(
118             report=report,
119             source_filename=file.filename,
120             output_path=pdf_path,
121         )
122
123         pdf_report_url = str(request.url_for("open_pdf_report", filename=
124             pdf_filename))
125
126     else:
127         file_hash = calculate_file_hash(temp_path)
128         existing_manual = db.query(Manual).filter(Manual.file_hash ==
129             file_hash).first()
130
131         if not existing_manual:
132             manual = Manual(
133                 manual_name=file.filename,
134                 fio_user=current_user.fio,
135                 department_code=current_user.id_department,
136                 faculty_code=current_user.id_faculty,
137                 file_hash=file_hash,
138             )
139             db.add(manual)
140             db.commit()
141
142     checked_file_url = str(request.url_for("download_checked_file",
143         filename=checked_filename))
144
145     return {
146         "message": "Проверка завершена",
147         "has_errors": has_problems,

```

```

143         "errors_count": sum(1 for issue in report.issues if issue.
144                               severity == "ERROR"),
145         "warnings_count": sum(1 for issue in report.issues if issue.
146                               severity == "WARNING"),
147         "checked_file_url": checked_file_url,
148         "pdf_report_url": pdf_report_url,
149     }
150
151 except Exception as e:
152     raise HTTPException(status_code=500, detail=f"Ошибка проверки
153     документа: {str(e)}")
154
155 finally:
156     file.close()
157
158 @router.get("/my")
159 def get_my_manuels(
160     current_user: User = Depends(get_current_user),
161     db: Session = Depends(get_db),
162 ):
163     manuals = db.query(Manual).filter(Manual.fio_user == current_user.fio).
164     all()
165
166     return [
167         {
168             "id_manual": m.id_manual,
169             "manual_name": m.manual_name,
170             "fio_user": m.fio_user,
171             "department_code": m.department_code,
172             "faculty_code": m.faculty_code,
173             "created_at": getattr(m, "created_at", None),
174         }
175         for m in manuals
176     ]

```

statistics.py

```

1 from datetime import date, datetime, time
2
3 from fastapi import APIRouter, Depends, HTTPException, Query
4 from sqlalchemy.orm import Session
5
6 from app.db.session import get_db
7 from app.models.manual import Manual
8 from app.models.user import User
9 from app.utils.dependencies import get_current_user
10
11 router = APIRouter(prefix="/statistics", tags=["statistics"])
12
13
14 def build_datetime_range(date_from: date, date_to: date):
15     start_dt = datetime.combine(date_from, time.min)
16     end_dt = datetime.combine(date_to, time.max)
17     return start_dt, end_dt
18

```

```

19
20 @router.get("/faculty")
21 def count_manually_by_faculty(
22     faculty_code: int = Query(...),
23     date_from: date = Query(...),
24     date_to: date = Query(...),
25     db: Session = Depends(get_db),
26     current_user: User = Depends(get_current_user),
27 ):
28     if date_from > date_to:
29         raise HTTPException(status_code=400, detail="date_from не может быть
30             больше date_to")
31
32     start_dt, end_dt = build_datetime_range(date_from, date_to)
33
34     count = db.query(Manual).filter(
35         Manual.faculty_code == faculty_code,
36         Manual.created_at >= start_dt,
37         Manual.created_at <= end_dt,
38     ).count()
39
40     return {
41         "entity": "faculty",
42         "value": str(faculty_code),
43         "date_from": date_from,
44         "date_to": date_to,
45         "manual_count": count,
46     }
47
48 @router.get("/department")
49 def count_manually_by_department(
50     department_code: int = Query(...),
51     date_from: date = Query(...),
52     date_to: date = Query(...),
53     db: Session = Depends(get_db),
54     current_user: User = Depends(get_current_user),
55 ):
56     if date_from > date_to:
57         raise HTTPException(status_code=400, detail="date_from не может быть
58             больше date_to")
59
60     start_dt, end_dt = build_datetime_range(date_from, date_to)
61
62     count = db.query(Manual).filter(
63         Manual.department_code == department_code,
64         Manual.created_at >= start_dt,
65         Manual.created_at <= end_dt,
66     ).count()
67
68     return {
69         "entity": "department",
70         "value": str(department_code),
71         "date_from": date_from,

```



```

71         "date_to": date_to,
72         "manual_count": count,
73     }
74
75
76 @router.get("/user")
77 def count_manually_by_user(
78     fio_user: str = Query(...),
79     date_from: date = Query(...),
80     date_to: date = Query(...),
81     db: Session = Depends(get_db),
82     current_user: User = Depends(get_current_user),
83 ):
84     if date_from > date_to:
85         raise HTTPException(status_code=400, detail="date_from не может быть
            больше date_to")
86
87     start_dt, end_dt = build_datetime_range(date_from, date_to)
88
89     count = db.query(Manual).filter(
90         Manual.fio_user == fio_user,
91         Manual.created_at >= start_dt,
92         Manual.created_at <= end_dt,
93     ).count()
94
95     return {
96         "entity": "user",
97         "value": fio_user,
98         "date_from": date_from,
99         "date_to": date_to,
100        "manual_count": count,
101    }

```

titlepage.py

```

1  import os
2
3  from fastapi import APIRouter, Depends, HTTPException
4  from fastapi.responses import FileResponse
5  from sqlalchemy.orm import Session
6
7  from app.db.session import get_db
8  from app.models.department import Department
9  from app.models.user import User
10 from app.schemas.title_page import TitlePageRequest
11 from app.service.title_page_generator import generate_title_page_docx
12 from app.utils.dependencies import get_current_user
13
14 router = APIRouter(prefix="/title -page", tags=["title -page"])
15
16
17 @router.post("/generate")
18 def generate_title_page(
19     data: TitlePageRequest,
20     current_user: User = Depends(get_current_user),

```

```

21 db: Session = Depends(get_db),
22 ):
23     print("TITLE PAGE ROUTER LOADED FROM:", __file__)
24
25     department = db.query(Department).filter(
26         Department.id_department == current_user.id_department
27     ).first()
28
29     if not department:
30         raise HTTPException(status_code=404, detail="Кафедра пользователя не
        найдена")
31
32     file_path = generate_title_page_docx(
33         manual_title=data.manual_title,
34         discipline_name=data.discipline_name,
35         audience=data.audience,
36         department_name=department.department_name,
37         department_code=department.department_code,
38         city=data.city,
39         year=data.year,
40     )
41
42     return FileResponse(
43         path=file_path,
44         filename=os.path.basename(file_path),
45         media_type="application/vnd.openxmlformats-officedocument.
        wordprocessingml.document",
46     )

```

admin.py

```

1 from fastapi import APIRouter, Depends, HTTPException, status
2 from sqlalchemy.orm import Session
3
4 from app.db.session import get_db
5 from app.models.department import Department
6 from app.models.faculty import Faculty
7 from app.schemas.admin import (
8     DepartmentCreate,
9     DepartmentResponse,
10    FacultyCreate,
11    FacultyResponse,
12 )
13
14 router = APIRouter(prefix="/admin", tags=["admin"])
15
16
17 @router.post("/faculty", response_model=FacultyResponse, status_code=status.
    HTTP_201_CREATED)
18 def create_faculty(data: FacultyCreate, db: Session = Depends(get_db)):
19     existing_faculty = db.query(Faculty).filter(
20         (Faculty.faculty_code == data.faculty_code) |
21         (Faculty.faculty_name == data.faculty_name)
22     ).first()
23

```

```

24     if existing_faculty:
25         raise HTTPException(
26             status_code=400,
27             detail="Факультет с таким названием или кодом уже существует"
28         )
29
30     faculty = Faculty(
31         faculty_name=data.faculty_name,
32         faculty_code=data.faculty_code,
33         dean_fio=data.dean_fio,
34         manual_count=0
35     )
36
37     db.add(faculty)
38     db.commit()
39     db.refresh(faculty)
40
41     return faculty
42
43
44 @router.post("/department", response_model=DepartmentResponse, status_code=
status.HTTP_201_CREATED)
45 def create_department(data: DepartmentCreate, db: Session = Depends(get_db)):
46     faculty = db.query(Faculty).filter(Faculty.faculty_code == data.
faculty_code).first()
47
48     if not faculty:
49         raise HTTPException(
50             status_code=404,
51             detail="Факультет с таким кодом не найден"
52         )
53
54     existing_department = db.query(Department).filter(
55         (Department.department_code == data.department_code) |
56         (Department.department_name == data.department_name)
57     ).first()
58
59     if existing_department:
60         raise HTTPException(
61             status_code=400,
62             detail="Кафедра с таким названием или кодом уже существует"
63         )
64
65     department = Department(
66         department_name=data.department_name,
67         department_code=data.department_code,
68         manual_count=0,
69         id_faculty=faculty.id_faculty
70     )
71
72     db.add(department)
73     db.commit()
74     db.refresh(department)
75     return department

```

Место для диска